

Benchmarks Can't See It

What quantization and speculative decoding really cost Qwen3.8-27B
in accuracy, context length and generation speed

Measured on two 16 GB consumer GPUs

Luiz Henrique Simões

Independent researcher

MSc candidate (professional master's), Data Science and Analytics, Universidade de São Paulo

simoeshz@gmail.com

September 2026

Abstract

Running a 27B-parameter model at home means choosing a quantization, a KV-cache precision, a context length, a GPU split and a speculative decoder, usually from tables measured on other hardware under an unstated protocol. This report measures those choices for Qwen3.8-27B on two 16 GB RTX 5060 Ti cards with 14 GiB of system RAM. *Accuracy.* Task benchmarks did not resolve an accuracy ordering between a 4-bit and a 6-bit build: HellaSwag ($n=400$), HumanEval+ ($n=164$), SWE-bench Verified ($n=50$), GPQA Diamond ($n=198$) and AA-LCR ($n=100$) return overlapping intervals, and paired HumanEval+ bounds the difference within about 4 points. Mean KL divergence from a 6-bit reference (2K tokens, 2.3 GPU-hours) orders every adjacent pair, in 30–32 of 32 paired windows. It is about twice as large on code as on prose and strongly right-skewed. It measures distance, not lost quality. Speculative decoding reproducibly changed 33 of 164 greedy completions without changing the observed task score. *Context.* The native 262,144-token window fits on 32 GB of consumer VRAM only after rebalancing the layer split by hand; the engine default fails for three of four builds, and the best split does not transfer between builds. The 4-bit build holds that window with an 8-bit cache, the 6-bit build only with a 4-bit cache; both drafters reach the same window unless the cache is unquantized. *Speed.* At the full window a smaller build is not measurably faster. On one engine build, a block-diffusion drafter (DFlash2) at its design depth decodes 45–65 % faster than the built-in multi-token-prediction head at every depth to 246K tokens, about $7\times$ faster than no speculation, with 3.6 GiB less host RAM. Host RAM, not VRAM, caused the only serving outage. Sixteen of the study's own claims were withdrawn or narrowed on re-analysis, fourteen at no GPU cost. All data, harness code and the decision log are public.

1 Introduction

A capable open-weight coding model now fits on hardware a person can buy. Qwen3.8-27B [1, 2] in a 4- to 6-bit GGUF build occupies 17–25 GB, and two 16 GB consumer cards hold it together with a long context. What that sentence hides is a stack of decisions: which quantization, which KV-cache precision, how much context, how to split layers between two cards that cannot share memory, which speculative decoder and how deep. Each is usually made from a published table, and each table was measured on different hardware, a different engine build, and a protocol the table rarely states.

Technical report. Not peer reviewed. Data, harness source and per-finding notes: <https://github.com/henrique-simoes/llm-quantization-damage>. Released under CC BY 4.0.

This report measures those decisions on one fixed machine: two NVIDIA RTX 5060 Ti 16 GB cards, a 12-thread desktop CPU and 14 GiB of RAM. The campaign ran from late August to mid-September 2026. It started as a practical question (*what is the best configuration for a local coding agent on this box?*) and turned into a methods question, because the first thing the measurements showed was that the usual instruments could not answer it.

The thesis. *The instrument decides whether there is anything to see.* Every task benchmark I ran on neighbouring quantizations of this model returned overlapping intervals: multiple choice, code generation, agentic software repair, graduate-level science questions and long-document reasoning, at the sample sizes those benchmarks actually have. Over the same builds, KL divergence against a reference build separated every adjacent pair with disjoint intervals, in about twenty minutes of GPU time per build. That is an independent confirmation of Dutta et al. [3], on a different model family, quantization scheme and hardware class. What this report adds is where the damage sits (a thin tail of tokens, far heavier on code than on prose), a measured bound linking the two kinds of instrument, and the same lesson repeated on the systems side: a context ceiling, a speed figure or a “lossless” claim is a property of a protocol as much as of a model.

The title’s six questions, in brief. The title names three costs (accuracy, context length, generation speed) of two choices (quantization, speculative decoding). The answers, each specific to this stack:

- *Quantization, accuracy.* Measurable on distributions, invisible on outcomes: mean KL divergence on code rises $3.7\times$ from the 6-bit to the 4-bit build, and no benchmark at $n = 30$ to 400 separates them (sections 7 and 10).
- *Quantization, context.* Three of four builds reach the native 262,144 tokens once the layer split is tuned; the largest stops at 212,992. At the full window the 4-bit build affords an 8-bit KV cache and the 6-bit build only a 4-bit one (section 8).
- *Quantization, speed.* Almost nothing at depth: the builds decode within 7 % of each other at the full window, inside the repetition spread (section 12).
- *Speculation, accuracy.* It changes 33 of 164 greedy completions, reproducibly, and moves no accuracy score beyond its interval (sections 9.1 and 10.2).
- *Speculation, context.* Both drafters fit the full window with a quantized cache; with an f16 cache the separate drafter costs one 32K-token rung more than the built-in head, and the built-in head at draft depth 8 no longer loads at 262K (sections 8.3 and 9).
- *Speculation, speed.* $3.4\text{--}4.2\times$ over plain decoding at 32K filled tokens, rising to $7.4\text{--}7.7\times$ at 246K; DFlash2 at its design depth is 45–65 % faster than MTP throughout (section 9.3).

What is in the report.

- **Quantization** (section 7). A KL-divergence ladder over four Unsloth Dynamic GGUF builds on prose, code and the actual task prompts; the tail structure of the damage; perplexity under three protocols, including one checkpoint that scored “29 % worse” or “0.8 % worse” depending only on convention; paired HellaSwag and HumanEval+ bounds; the cost of a 4-bit KV cache.
- **Context** (section 8). Exact KV arithmetic for this hybrid architecture; how a manual layer split moves the reachable window from failing to the native 262,144 tokens; a bracketed

context $\times$ KV-precision map for two builds and two drafters; three distinct ways a long-context server fails; and a host-RAM incident that no VRAM measurement could have predicted.

- **Speculative decoding** (section 9). Evidence that speculation is a reproducibly different decode path, not a free speed knob; a like-for-like comparison of the model’s built-in multi-token-prediction (MTP) head against the DFlash2 block-diffusion drafter on one engine build, at depths up to 246K tokens.
- **Accuracy versus depth** (section 10). RULER retrieval to 131K tokens, including a headline result I withdrew after finding it measured output budget, not retrieval; AA-LCR long-document reasoning and GPQA Diamond under three independent judges.
- **Agentic behaviour, speed, energy** (sections 11 and 12), the resulting **configuration** (section 13), a **register of the study’s own defects and retractions** (section 14), and **limitations** (section 15).

Status of this document. This is a technical report by one independent practitioner. It is not peer reviewed and does not come from a university group or an AI lab. I am a professional master’s candidate at the Universidade de São Paulo; the university did not sponsor or review this work. One model, one machine, one engine family: the numbers are specific to this stack and the report says so wherever it matters. What should transfer is the method and the failure modes. Every number traces to a committed artifact through a per-finding note, and the notes that turned out wrong are still there, marked as superseded.

How to read it. Readers who want the configuration can go to section 13. Readers choosing a quantization should read section 7; readers fighting for context length, section 8; readers choosing a drafter, section 9. Each table states its protocol, n and estimator in the table. Rows marked ^H were measured before 2026-08-29 on an engine image that no longer exists and cannot be reproduced on current builds (section 14); they are never mixed with later rows. Blue boxes give the one-sentence takeaway of a section.

2 Background and related work

Weight quantization in the GGUF ecosystem. llama.cpp [4] stores weights in block-quantized “k-quant” formats: a tensor is cut into small blocks, each with its own scale, at nominal widths from 2 to 8 bits. Community builds mix widths per tensor. The Unsloth Dynamic (UD) builds used here [5, 6] keep sensitive tensors at higher precision and calibrate with an importance matrix, so “UD-Q4_K_XL” names a recipe, not a uniform 4-bit model. NVIDIA’s NVFP4 [7] is a 4-bit floating-point format with hardware support on Blackwell GPUs; it is served here through vLLM [8, 9].

Measuring quantization damage. The usual report is perplexity on WikiText-2 [10] plus a battery of task benchmarks. Both have known weaknesses. Perplexity is an average, and practitioners have long noted that the average hides token-level movement [11]; llama.cpp therefore ships a KL-divergence mode that compares full output distributions against saved reference logits [12]. Dutta et al. [3] made the general argument rigorously: accuracy can stay flat while a large share of individual answers “flip”, and they recommend KL divergence and flip counts as the metrics of record. Public tables by Unsloth [5], Fireworks [13], LocalBench [14] and others [15] report KL divergence, nearly always on prose and at about 2K tokens of context.

Fireworks suggests mean KLD below 0.007 as a deployment-quality threshold; I cite it as an outside reference point, not as a pass mark. On statistics, I follow Miller [16] and Bowyer et al. [17]: report intervals, analyse paired designs as paired, and distrust normal approximations at small n .

Long context. RULER [18, 19] generalises needle-in-a-haystack tests with multi-key, tracing and aggregation variants. HELMET [20] and NoLiMa [21] show that literal-match retrieval overstates long-context ability. LongPPL [22] argues that perplexity must be restricted to context-dependent tokens to mean anything at length. Two studies measure quantization at long context directly and find 4-bit weights lose measurable accuracy at long context while 8-bit is nearly free [23, 24]. Artificial Analysis’ Long Context Reasoning set (AA-LCR) [25] asks questions that require reasoning over roughly 100K tokens of real documents. For KV-cache quantization, KIVI [26] and KVQuant [27] establish that 4-bit KV is near-lossless only with per-channel key quantization, which llama.cpp’s block-wise q4_0 cache does not do; a community measurement on a sibling model reports KV-induced divergence one to two orders of magnitude larger at 250K tokens than at 2K [28].

Speculative decoding. A cheap drafter proposes several tokens and the target model verifies them in one pass [29, 30]. Under exact arithmetic the accepted output is distributed identically to the target’s own. Drafters differ: Medusa adds extra heads [31]; multi-token prediction (MTP) trains the predictor with the model [32, 33], and Qwen3.8 ships with such a head; DFlash [34] is a small block-diffusion drafter that proposes a whole block in parallel and conditions on hidden states tapped from the target. The Qwen3.8 drafter of that family is referred to here as DFlash2 [35, 36]. “Lossless” holds only up to floating-point numerics: GPU kernels can be deterministic run-to-run and still give different results at different batch shapes [37], vLLM documents the same caveat [38], and an open llama.cpp issue reports greedy divergence under speculation [39].

Benchmarks used. HumanEval [40] with the EvalPlus extended tests [41]; HellaSwag [42]; SWE-bench [43] in its human-validated Verified subset [44], driven by mini-SWE-agent [45, 46]; GPQA Diamond [47]; RULER; AA-LCR. Qwen’s card reports LiveCodeBench v6 [48], SWE-bench Pro and Terminal-Bench [49] for this model and no HumanEval figure; none of those three was run here, so no number in this report is comparable to the model card (section 15).

What is new here, and what is not. This is a measured case study of one deployment, not a new evaluation principle. That accuracy can mask distributional change is prior work [3]. That perplexity depends on window, stride and tokenization is documented [50]. Community tools already report token-level quantiles and tails of KL divergence alongside the mean [51], including for this model [52]. Greedy divergence under speculation on quantized targets has been reported as a bug [39]. What this report contributes are specific measurements I could not find elsewhere: the code-against-prose comparison of the divergence distribution with a cache-only control; a paired interval placing a $3.7\times$ divergence increase beside a pass@1 difference within about four points; one 36-fold perplexity swing decomposed into its conventions; a bracketed context map for a hybrid-attention model on two unlinked consumer cards; a benchmark-scale count of speculative non-identity with a determinism control; and a same-build MTP versus DFlash2 comparison to 246K tokens of filled context.

Table 1: The test machine. Everything in this report was measured on this one host.

GPUs	2 × NVIDIA GeForce RTX 5060 Ti 16 GB (Blackwell, compute capability 12.0), 180 W power limit each, 16,311 MiB usable per card, no NVLink, no peer-to-peer path
GPU attachment	GPU0 on a CPU PCIe root port; GPU1 behind the chipset bridge, sharing its uplink with the SATA controller (both disks and the swap file), the network interfaces and USB. Both links train at x4 on x8-capable cards
CPU / RAM	AMD Ryzen 5 8500G, 6 cores / 12 threads; 14 GiB usable system RAM; 4 GiB swap (12 GiB during the second battery, section 14)
Storage	Two SATA disks: system (218 GB) and a separate 110 GB model store
Driver	NVIDIA 595.84, CUDA 13.2; Ubuntu Linux; all engines run in Docker containers
Power sensing	GPU power from the driver at 1 Hz; CPU package from RAPL; <i>no</i> wall-socket, BMC or IPMI sensor. “System” power anywhere in this report is modelled, not measured

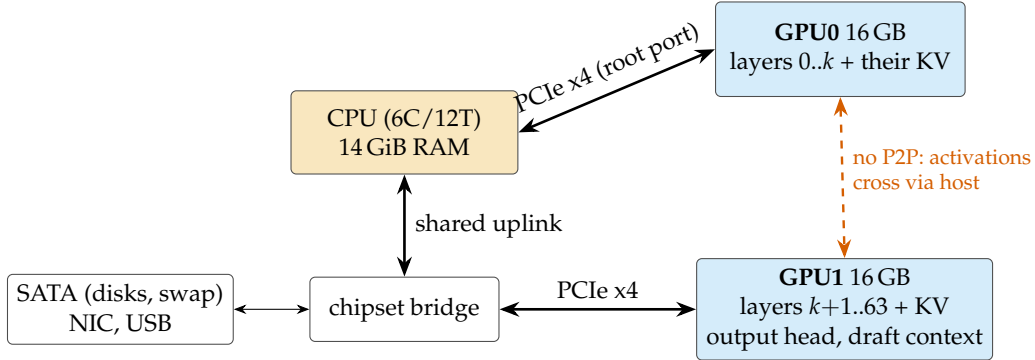


Figure 1: The two cards are not a memory pool and are not attached symmetrically. With layer split, each layer’s weights *and its slice of the KV cache* live on one card, so a configuration fails when the fuller card runs out, however much is free on the other. GPU1 also carries the output head and the speculative draft context, and sits behind the shared chipset uplink.

3 Hardware

[Table 1](#) lists the machine; [figure 1](#) shows the part of it that matters most. Three properties shape everything that follows.

The cards are two 16 GB budgets, not one 32 GB budget. There is no NVLink and the driver reports no peer-to-peer path. Of the split modes llama.cpp offers, only layer split (`-sm layer`) runs on the engine builds that survive ([section 5](#)); it places whole layers, with their KV slices, on one card or the other. The binding limit is therefore 16,311 MiB on whichever card is fuller. [Section 8](#) shows default placement stranding 2–3.8 GiB on GPU0 while GPU1 ran out.

The cards are attached asymmetrically. GPU1 shares a chipset uplink with both disks, the swap file and the network. I did not test whether this contributes to the per-card asymmetries measured later; the documented cause of those is layer placement. It is recorded because a host description that omits it would be incomplete.

System RAM is small, and it binds twice. With the model memory-mapped, about 10 GiB is available. This (i) caps the standard KL-divergence tool at 8,192 tokens of context, because it holds a chunk’s logits in memory at 248 KB per token (at 16,384 tokens the host fell to 305 MiB free and wrote nothing in 13 minutes), and (ii) caused the study’s only serving outage ([section 8.5](#)). No full-precision reference fits anywhere: BF16 weights are 54.7 GB, so every divergence in this report is measured against the largest quantized build, not against the original model.

4 Software versions and model builds

Tables 2 and 3 pin the stack. Two facts about versions turned out to matter more than expected.

Deleting one image froze the exploratory period’s results. Build E-H was the only one on which tensor split (`-sm tensor`) ever worked on these cards. On E-A and E-B the flag is listed in `-help` and the server dies with a CUDA illegal-memory-access error. When E-H was deleted during routine disk cleanup, every tensor-split result and every 262K-token result measured until then became unreproducible, and two model files from that period were also gone. Re-measuring on surviving builds gave *different* context ceilings. Those early rows are kept, marked ^H, and never placed in a table with later ones. *Flag availability is not capability.*

A drafter file is bound to an engine build. The DFlash2 drafter loads only on builds that know its tensor layout. On E-A it fails with wrong number of tensors; expected 81, got 58, and a harness that scored by exit status recorded a clean 0.000 pass@1, indistinguishable in a results table from a model that ran and failed. The same signature later blocked a production cut-over (section 13). The drafter file itself was also revised upstream during the study (a rope-metadata fix); the first-battery DFlash2 rows used the earlier revision on a different engine from the MTP rows, which is why section 9 ranks the drafters only from the second battery.

Engine defaults are not what the documentation says. On E-A, a request with no sampling fields runs at temperature 1.0, top-*k* 20, top-*p* 0.95, min-*p* 0.05. That matches neither llama.cpp’s documented defaults (0.8 / 40 / 0.95 / 0.05) nor either of Qwen’s presets. The server echoes effective settings per request, so this is cheap to check; every run here sends an explicit sampling block and asserts the echo. The model also *reasons by default*. The widely quoted `/no_think` prompt suffix does not disable it on this chat template. What does is the request field

```
chat_template_kwargs: {"enable_thinking": false}
```

while `{"reasoning_effort": "none"}` raises a template exception (accepted values are low, medium, xhigh). A two-token probe distinguishes the states in about a second and ran before every battery.

5 Three inference servers, and why llama.cpp

I tried three servers before settling on one: llama.cpp with GGUF builds, vLLM with NVIDIA’s NVFP4 checkpoint, and SGLang. This section reports what each could do on this hardware. All of it predates 2026-08-29 (^H), so it is orientation, not a controlled comparison, and the comparison it does support is narrower than it looks: backend, weight format, KV precision and speculative method all change together between rows.

vLLM with NVFP4. Tensor parallelism splits every layer across both cards, so vLLM balances memory better than llama.cpp’s layer split does. It lost on two other axes. *Context*: at the highest GPU-memory-utilisation setting that boots (0.97; at 0.99 the server dies within a minute on 16 GB cards) the nightly build reports a KV pool of 52,337 tokens, a working ceiling of 51,200, about one fifth of the model’s native window, and MTP takes a little more. An earlier run on the stable image recorded 98,304; the two were never measured back to back, so the ceiling may be image-dependent. Neither figure was verified with a deep prefill, because that battery was stopped early. *Fidelity*: NVFP4 scored 85.4/84.1 on HumanEval/HumanEval+ in thinking mode against 90.9/88.4 for Q6_K_XL (table 11); on matched perplexity windows it sits just below the whole GGUF ladder (section 7.4); and Unsloth’s published KL divergence for it on code is

Table 2: Engines and tools. A result is tied to the image that produced it; image digests are in the repository’s `env-manifest.json`.

label	version	used for
E-A llama.cpp (MTP build)	0.3.0-dev, commit d222767; image feb0231976b6	first battery: divergence, HellaSwag, HumanEval+, RULER, split sweeps, MTP speculation
E-B llama.cpp DFlash fork	0.1.2-dev build 50, commit f7aadeb; image 22bb8b7fed8b	the only DFlash2-capable build during the first battery
E-C llama.cpp upstream	build b10975, commit 4c9233c; image 5268283a8d65	second battery: both drafters on one build; KV map, speed, AA-LCR, GPQA, soak; now serving
E-H llama.cpp PR #27342 build	commit 1deefcc (<i>image deleted</i>)	all measurements before 2026-08-29, marked ^H
vLLM	0.27.1 (stable) and a 0.26.1rc1 nightly	NVFP4 accuracy; NVFP4 speculative speed
SGLang	0.5.18	attempted; never started (section 5)
EvalPlus	patched local install	HumanEval / HumanEval+ scoring
mini-SWE-agent	2.4.6	SWE-bench Verified agent scaffold
RULER	reference generators and metric	long-context retrieval

Table 3: Model builds. All are Unsloth Dynamic GGUFs of Qwen3.8-27B and all embed the MTP head. Q6_K_XL is the divergence reference. Sizes in bytes as downloaded; SHA-256 prefixes are in the repository.

build	role	bytes	GiB	batteries
Q6_K_XL	divergence reference	25,299,061,664	23.56	first
Q6_K	arm	21,983,677,344	20.47	first, second
Q5_K_XL	arm	20,876,938,144	19.44	first
Q4_K_XL	arm	17,559,178,144	16.35	first, second
IQ4_XS	historical only	14,252,845,984	13.27	^H
Q3_K_XL, Q4_K_M	historical only (files deleted)	—	—	^H
NVFP4 (W4A4, FP8 KV)	vLLM checkpoint	≈22 GB	—	^H
DFlash2 drafter Q4_K_M	speculative sidecar	1,143,006,816	1.06	second

0.026 against roughly 0.002 for their 6-bit dynamic build [5]. None of those gaps is large, and the HumanEval+ one is inside its interval. Taken together with a five-fold context deficit they decided the practical question.

The larger speculative multiplier belongs to the slower baseline. In [table 4](#) vLLM gains $3.3\times$ from MTP and llama.cpp gains $1.4\text{--}1.7\times$, yet the best absolute rates differ by 7%. The `-enforce-eager` setting this configuration needs slows vLLM’s plain decode to 12.4 tok/s, which leaves more for speculation to win back. A speed-up ratio without its baseline says little. One thing does carry across engines: MTP acceptance at depth 2 is 0.728 on llama.cpp and 0.709 on vLLM, which suggests acceptance is mostly a property of the model’s draft head, not of either implementation.

A separate draft model hits a wall on 16 GB cards. SGLang failed at every context length tried (131K down to 16K) with Tried to allocate 1.19 GiB while initialising its EAGLE draft worker. vLLM with the 3.6 GB BF16 DFlash2 drafter failed on an allocation of the *same size* at every utilisation setting from 0.78 to 0.97. llama.cpp with a 1.1 GB 4-bit GGUF drafter worked. So three stacks were tried, two failed the same way, and the one that succeeded differs in

Table 4: One probe across engines^H: greedy, 1,024-token generation, single stream, median of three runs, **decode into a nearly empty KV cache**. vLLM runs tensor-parallel across both cards with `-enforce-eager`. Power is GPU telemetry plus CPU package plus a fixed platform allowance (modelled). Context ceiling is the largest window that boots (vLLM) or that prefills and generates (llama.cpp); the llama.cpp ceilings are first-battery values from [section 8](#), the only non-historical cells in this table, shown for orientation.

backend / weights	speculation	tok/s	speed-up	accept.	avg W	J/tok	ctx ceiling
vLLM / NVFP4	none	12.44	1.00×	—	126	9.96	51,200
vLLM / NVFP4	MTP $n=2$	28.11	2.26×	0.709	133	4.57	≈49K
vLLM / NVFP4	MTP $n=4$	41.21	3.31×	0.559	148	3.60	≈49K
vLLM / NVFP4	DFlash2 (BF16 drafter)	infeasible: out of memory at every setting tried					
llama.cpp / Q4_K_XL	none	23.00	1.00×	—	207	9.00	262,144
llama.cpp / Q4_K_XL	MTP $n=2$	32.69	1.42×	0.728	214	6.55	262,144
llama.cpp / Q4_K_XL	DFlash2 $n=4$ (4-bit drafter)	38.51	1.67×	0.714	218	5.67	262,144
SGLang 0.5.18	EAGLE	never started: out of memory initialising the draft worker					

the two ways one would predict: a quantized drafter, and layer-split placement that leaves per-card headroom where tensor-parallel replication does not. The matching 1.19 GiB figure is an observation; no allocator was instrumented. MTP avoids the problem on both backends because its predictor lives inside the checkpoint. SGLang produced no measurement of any kind, and nothing here says anything about its quality: a non-speculative SGLang configuration was not attempted.

Why llama.cpp. On two unlinked 16 GB cards it was the only server that reached the native 262K window, the only one that ran a separate drafter, and the one with the wider choice of weight precisions. vLLM’s tensor parallelism balances memory better and its MTP path was the most energy-efficient configuration measured (3.60 J/token at depth 0), so this is a statement about this memory budget and these builds, not about the engines in general.

6 Method

6.1 Two batteries

The work came in two batteries plus an earlier exploratory period.

Exploratory period^H (2026-08-20 to 08-28). Perplexity, HumanEval+ in two modes, SWE-bench Verified, agentic step counts, cross-backend speed. Greedy decoding throughout. Measured on the deleted build E-H.

First battery (08-29 to 09-03), build E-A. Four builds: Q4_K_XL, Q5_K_XL, Q6_K and the reference Q6_K_XL. Context ceilings under a layer-split sweep; KL divergence on three corpora; HellaSwag; paired generative HumanEval+; speculative equivalence and determinism; RULER to 131K.

Second battery (09-15 to 09-19), build E-C. Two builds (Q4_K_XL, Q6_K) × two drafters (MTP, DFlash2) on one upstream build: a bracketed context × KV-precision map, a matched-depth speed probe, AA-LCR and GPQA Diamond at the model’s highest reasoning effort, and a 252K-token multi-turn soak. Its design was fixed in a decision record before it ran, including the rule for choosing a drafter.

6.2 Instruments

KL divergence. `llama-perplexity -kl-divergence` against saved logits from Q6_K_XL, context 2,048, 32 chunks (65,536 tokens) per cell, on (a) WikiText-2 test, (b) a Python corpus built from Django source files, (c) the 164 HumanEval+ prompts (18,432 tokens). The tool reports mean KLD with a standard error that treats tokens as independent, top-1 agreement, and quantiles of per-token KLD without intervals. Divergence here is *ladder-relative* (distance from the 6-bit reference, whose own damage is unmeasured and zero by construction) and is computed on *prompt tokens* under teacher forcing, so it is not a measurement of generated-code quality.

Task benchmarks. HellaSwag through the same tool, 400 items, identical items for every build. HumanEval+ through EvalPlus, 164 problems, scored on base and extended tests. SWE-bench Verified, 50 instances, mini-SWE-agent with a 250-step limit, scored from per-instance reports on an x86-64 host. RULER single-needle and multi-key retrieval from RULER’s own generators and `string_match_all` metric. GPQA Diamond (198 items) with Artificial Analysis’ prompt template and last-match answer extraction [53], cross-checked against the `simple-evals` extraction pattern [54]; the two agreed on all 496 scored answers. AA-LCR (100 questions over 30 document sets of about 100K tokens) with its published equality-checker prompt, judged by three different LLM judges (section 10.2).

Context ceiling. A window counts as reached only if the server (i) loads with `-fit off` (otherwise `llama.cpp` may silently shrink the context), (ii) reports the requested `n_ctx`, (iii) prefills at least 94 % of it with real source code, and (iv) then generates a fixed number of tokens. VRAM is read after the deep prefill, because compute buffers grow with batch; it is recorded, never used as a pass mark (an early VRAM threshold rejected a rung that works). A ceiling needs a failed rung above it, attempted twice, because layer-split memory varies by 100–200 MiB from load to load.

Speed. Decode rate is the server’s own `predicted_per_second` over a generation of known length. Decode into an empty cache and decode at depth differ by 3–5× on this host, so every speed figure states its filled depth. A harness rule added after a failure (section 14): a speed cell is valid only if the generation reached its requested length.

6.3 Sampling configurations

Table 5: Sampling configurations. Absolute scores are not comparable across rows, and rows are never mixed in a table.

configuration	temp	top- <i>p</i>	top- <i>k</i>	min- <i>p</i>	presence	used for
engine default as measured	1.0	0.95	20	0.05	0	nothing (a hazard, section 4)
Qwen official, non-thinking	0.7	0.80	20	0	1.5	split sweeps, paired HumanEval+
Qwen official, thinking	1.0	0.95	20	0	0	AA-LCR, GPQA (effort xhigh)
greedy	0	1	—	—	0	divergence, HellaSwag, equivalence, speed probe, ^H rows
greedy, reasoning on	0	1	—	—	0	RULER only

6.4 Statistics

Pass rates carry Wilson 95 % intervals [55]. Two builds scored on the same items are compared as paired data: the difference in proportions $d = (b - c)/n$ over the discordant counts b, c , a Tango score interval on d (a Wald interval is unreliable with three to five discordant pairs), and

an exact McNemar test [56]. Alongside a null I state the smallest p the test could have returned *given the observed* number of discordant pairs (it is 2^{1-m} for m pairs; a property of the data in hand, not a power calculation); with three discordant pairs that minimum is 0.25, so “ $p = 1.0$ ” carries no information about the builds. Continuous summaries use bootstrap intervals [57]. Speed spreads are $(\max - \min) / \text{median}$ over repetitions matched on every launch parameter. The unit of independence for speculative acceptance is the generation, not the draft event: pooled per-event intervals of ± 0.02 sat on run-to-run ranges of ± 0.3 . Nineteen tests across the first battery are corrected together in [section 12.4](#).

6.5 How the work was run

An agent-driven harness executed the campaign under my direction. I designed the protocol, approved each experiment before it ran, and am responsible for all content including every error. AI assistants also helped analyse artifacts and draft this text. The decision log, a ledger of what was run, and about a hundred per-finding notes (each with its evidence path and caveats) are released with the data, so the chain from an instruction to a number can be inspected. Independent adversarial review passes over those notes produced most of the corrections in [section 14](#). Operating rules that earned their place: server logs are saved before any container is removed; nothing is deleted before its hash is recorded; every battery starts with a pilot that must pass; a run that measured nothing must not exit successfully; notes are appended and superseded, never edited.

7 Quantization: choices and tests

7.1 Which builds, and why

The ladder is four Unsloth Dynamic builds between 16 and 24 GiB ([table 3](#)). The choice was bounded from both sides. Below it, the 3-bit build was the fastest configuration measured in the whole study and failed every agentic task it was given ([section 11](#)). Above it, an 8-bit build (31 GB) leaves no room for context and BF16 does not fit at all. Q4_K_XL is Unsloth’s recommended default for this model; Q6_K_XL is the largest build that still leaves a usable window, which also makes it the only available reference. Two further points, IQ4_XS and NVFP4, appear in the historical rows. Six intermediate builds in the repository (Q4_K_S, Q5_K_S, Q5_K_M, Q6_K_M, Q6_K_L and the deleted Q4_K_M) were not tested.

7.2 Divergence ranks the ladder

[Figure 2](#) is the central result. Against the Q6_K_XL reference, mean KLD on code is 0.0058 ± 0.0002 for Q6_K, 0.0103 ± 0.0005 for Q5_K_XL and 0.0215 ± 0.0008 for Q4_K_XL. The order $Q6_K < Q5_K_XL < Q4_K_XL$ holds in every domain ([table 6](#)). Each cell took about twenty minutes. The full table with quantiles is in [section A](#).

Three things follow.

Prose tables flatter the quantizer. The same build diverges $1.8\text{--}2.6\times$ more on code than on prose, and $3.1\text{--}4.4\times$ more on the HumanEval+ prompts than on prose. The amplification grows as bits are removed (prose to task: $3.1\times$, $3.9\times$, $4.4\times$). Against the external 0.007 line, two of three builds pass on prose, one passes on code, none passes on the task prompts. Public quantization tables are almost all measured on prose.

Top-1 agreement points the other way. On code every build agrees with the reference on the argmax token *more* often than on prose (98.4–99.1 % against 96.2–97.5 %), while its mean

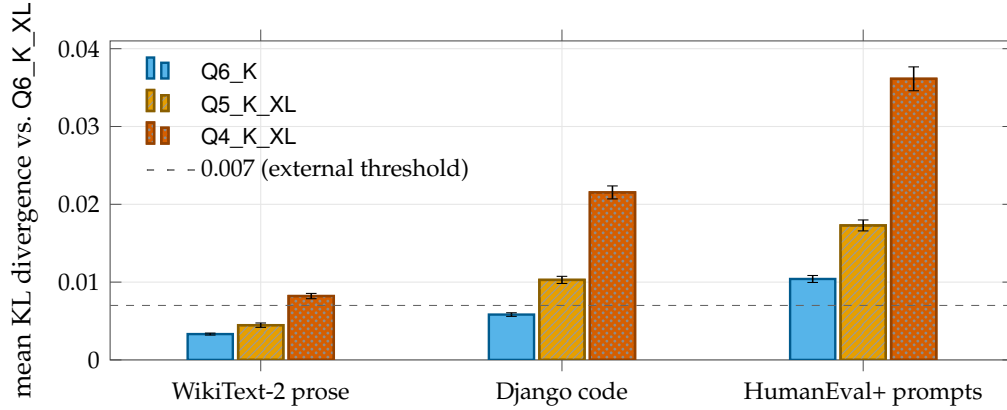


Figure 2: Divergence separates every adjacent pair of builds in every domain, and the damage grows as the corpus approaches the real task. Bars are mean KLD ± 1 tool-reported SE; $n = 65,536$ tokens per cell (18,432 for task prompts); context 2,048; q4_0 KV; greedy teacher forcing. The dashed line is an external quality threshold, shown for orientation only.

KLD is double. Code is far more predictable (reference perplexity 1.18 against 5.79), so the top token usually survives even as the distribution under it moves. A table reporting only top-1 agreement would conclude this model quantizes better for code. Report the pair. On the task prompts Q4_K_XL agrees on 95.9 % of tokens: about one token in 24 differs from the reference under greedy decoding.

How sure is the ordering? The tool’s standard error treats 65,536 tokens from contiguous windows as independent, and comparing two builds through their separate errors ignores that they were scored on the same tokens against the same reference logits. Both problems have a fix that costs no GPU time. The tool prints a running mean after each 2,048-token window, so per-window means can be recovered by differencing, and the builds can be compared *paired by window* (table 6). Every adjacent difference has a bootstrap interval that excludes zero, and the larger build has the lower divergence in 30 to 32 of 32 windows. The naive calculation had given 8.7–18 “standard errors” on code; the paired window-level t is 4.1–4.5 there, and 5.5–11 on prose. The ordering is secure. The size of the naive separations was not.

What this instrument measures. Divergence here is a distance: the mean teacher-forced KL from one 6-bit reference, on these corpora, at 2,048 tokens of context. It is not accuracy, and a larger distance is not by itself a worse model; Dutta et al. [3] keep distance metrics and capability metrics apart for the same reason. KL divergence obeys no triangle inequality, so an ordering of distances from Q6_K_XL does not establish the ordering of distances from the unquantized model, and errors that the reference shares with a build would make that build look closer than it is. The builds also mix bit-widths per tensor, so the ordering describes three recipes, not a clean effect of bit count. I use “damage” in this report as shorthand for this distance, and the task benchmarks below are the only evidence here about whether it matters.

7.3 The damage lives in a thin tail

Figure 3 divides each quantile of per-token KLD on code by the same quantile on prose. About 90 % of code tokens are nearly untouched by quantization; a band between the 95th and 99.9th percentile moves a great deal. A mean-only report averages “almost nothing happens” with “something drastic happens” and states neither.

This is a candidate explanation for why benchmarks see so little. A benchmark scores outcomes, not tokens. A perturbation confined to a few percent of positions changes an outcome only when one of those positions is decisive. That predicts small discordance between builds,

Table 6: Paired differences in mean KLD between builds, by window. Per-window means recovered from the tool’s running mean (printed to five decimals); percentile bootstrap over windows, $B = 20,000$. Windows are contiguous chunks of one corpus per domain, so the intervals cover window-to-window variation on these corpora, not the choice of corpus.

corpus (windows)	pair	mean difference	bootstrap 95 % CI	windows where larger build is closer
code (32)	Q5_K_XL – Q6_K	0.00446	[0.0027, 0.0065]	32 / 32
	Q4_K_XL – Q5_K_XL	0.01124	[0.0063, 0.0170]	31 / 32
prose (32)	Q5_K_XL – Q6_K	0.00114	[0.0008, 0.0016]	30 / 32
	Q4_K_XL – Q5_K_XL	0.00375	[0.0032, 0.0045]	32 / 32
task prompts (9)	Q5_K_XL – Q6_K	0.00689	[0.0056, 0.0081]	9 / 9
	Q4_K_XL – Q5_K_XL	0.01884	[0.0160, 0.0215]	9 / 9

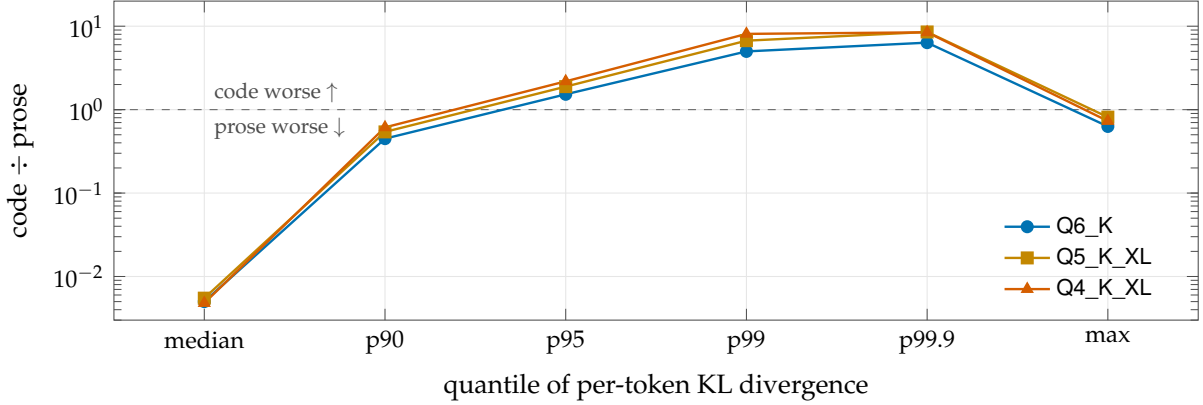


Figure 3: “Code is twice as bad as prose” is an average of two opposite facts. At the median a code token is perturbed about $200\times$ less than a prose token; the order flips between the 90th and 95th percentile; at the 99th, code is $5\text{--}8\times$ worse. The single worst token is again milder on code. Quantiles carry no interval, and the median row cannot rank the builds (its inputs have one or two significant digits).

and small discordance is what the paired benchmarks below show (3 and 5 problems of 164). It is a hypothesis with one successful quantitative prediction; no experiment here manipulates the tail and watches a benchmark move.

A scope I got wrong at first. I initially read the tail as something quantization *does*. The study contains a control that says otherwise. One cell perturbs only the KV cache (q4_0 against f16) on the reference weights. Normalising each cell by its own mean, the 99th percentile sits at $24\text{--}26\times$ the mean for all three quantized builds on code, at $23\times$ for the KV-only cell, and at $8.4\text{--}8.6\times$ for every build on prose (table 7). The *shape* of the tail belongs to the corpus. Quantization sets the *level*: $0.0030 \rightarrow 0.0058 \rightarrow 0.0103 \rightarrow 0.0215$. The control was not designed for this purpose, so the comparison is suggestive, not clean.

7.4 Perplexity: right order, no resolution, and a 36-fold swing

Perplexity orders the ladder correctly (table 8) and cannot certify a single step of it: even the extremes overlap. A second protocol from the same period (20 windows of 4,096 tokens, f16 KV) gave different absolute values (5.50–5.55) and was not even monotone. The KV-cache experiment makes the same point from the other side: switching the cache from f16 to q4_0 moved code perplexity from 1.1791 to 1.1809 (+0.15 %) while the measured KL divergence of that change is half that of dropping a whole quantization level.

Figure 4 is the most transferable result of the exploratory period. The first NVFP4 perplexity (8.58 against 6.65) suggested a 29 % loss. Re-scored on the same windows under the same half-window rule it is 6.71: a 0.8 % loss, a 36-fold change in the estimated effect from convention

Table 7: Tail shape, each cell normalised by its own mean KLD. Flat within a corpus over a $7\times$ range of mean divergence; different by about $3\times$ between corpora; reproduced by a perturbation with no weight quantization in it.

cell	corpus	perturbation	mean KLD	median/mean	p95/mean	p99/mean
Q6_K/ Q5_K_XL/ Q4_K_XL	prose	weights	.0033 / .0045 / .0082	0.41–0.43	3.1–3.2	8.4–8.6
Q6_K/ Q5_K_XL/ Q4_K_XL	code	weights	.0058 / .0103 / .0215	0.001	2.6–2.8	24.4–26.2
Q6_K_XL	code	KV dtype only	.0030	0.001	2.8	22.9
Q6_K/ Q5_K_XL/ Q4_K_XL	task prompts	weights	.0104 / .0173 / .0361	0.02	4.8–4.9	17.1–18.3

Table 8: WikiText-2 perplexity^H, Protocol 1: 602 windows of 512 tokens, scoring the second half of each window with the first half as context; $\pm$ is the tool’s standard error. The whole ladder spans 0.033 while each point carries ± 0.041 .

build	PPL	$\pm$ SE	Δ vs best
Q6_K_XL	6.6511	0.0411	—
Q5_K_XL	6.6556	0.0411	+0.07 %
Q4_K_XL	6.6617	0.0412	+0.16 %
IQ4_XS	6.6839	0.0413	+0.49 %
NVFP4 (vLLM), re-scored on the same windows	6.7073	n/r	+0.85 %

alone. The corrected number still ranks NVFP4 below every GGUF build, so the ranking held and the magnitude did not. A perplexity published without its corpus file, window count and scoring rule cannot be compared with another.

7.5 What task benchmarks can say: bounds

Tables 9 and 10 show the two first-battery task benchmarks on the same builds that divergence orders in every window. HellaSwag spreads the four builds over one point, puts the 4-bit build on top, and finds two builds identical on every item. HumanEval+ has the ladder’s extremes agreeing on 159 of 164 problems.

My first write-up reported the HumanEval+ result as “McNemar $p = 1.0$, indistinguishable”. That test could not have reached significance: with three discordant pairs the smallest possible two-sided p is 0.25. This is the very error the report warns about, and I made it in the report’s own anchor experiment. The informative statement is the interval:

A $3.7\times$ increase in code-prompt KL divergence (Q6_K_XL $\rightarrow$ Q4_K_XL) moves HumanEval+ pass@1 by -0.6 points, with a 95 % confidence interval reaching about 4 points. Benchmarks at these sample sizes *bound* quantization damage; they do not *rank* it.

The historical HumanEval+ ladder (table 11) agrees. Scores rise with bit-width in the same order as perplexity and divergence, yet the three upper builds span 3.7 points against a ± 4.6 -point interval; the 5-bit to 6-bit step is one problem. Two side results are worth keeping. The context control is the cleanest null in the study: allocating a 131K window instead of 32K changed nothing, to the last problem. And in thinking mode with a 4,096-token budget, three of four MTP rows score exactly 86.0 on HumanEval+ and differ only in their empty-response rate (12.8 % down to 7.9 % with fidelity): that benchmark partly measures whether reasoning fits its budget. The same mechanism returns in section 10.1.

7.6 The 4-bit KV cache is validated at 2K tokens, and only there

Every context result in this report depends on a quantized KV cache, so its cost was measured: q4_0 against f16 on the reference build, code corpus, gives mean KLD 0.00296 ± 0.00013 with

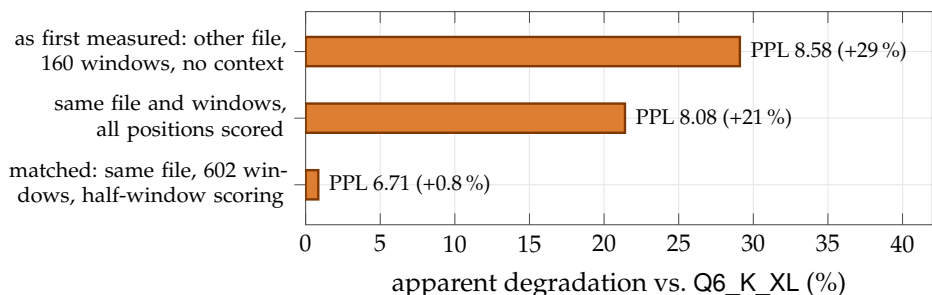


Figure 4: One checkpoint, one benchmark corpus, three conventions^H. The NVFP4 build looked 29 % worse than the GGUF ladder until it was re-scored on the ladder’s exact windows with the ladder’s scoring rule. The corpus file (two conversions of the same dataset, 297K against 309K tokens), the number of windows and the scoring rule all contributed; tokenizer mismatch was suspected, tested and ruled out.

Table 9: HellaSwag, first battery. 400 items, the same items for every build, greedy log-likelihood scoring. The most quantized build scores nominally highest; two builds answer all 400 items identically.

build	correct	acc. %	Wilson 95 %	pair	discordant	diff. (pts)	min. attainable <i>p</i>
Q6_K_XL	331	82.75	[78.7, 86.1]	Q6_K_XL vs Q6_K	2	+0.50	0.50
Q6_K	329	82.25	[78.2, 85.7]	Q6_K_XL vs Q5_K_XL	0	0.00	1.00
Q5_K_XL	331	82.75	[78.7, 86.1]	Q6_K_XL vs Q4_K_XL	2	−0.50	0.50
Q4_K_XL	333	83.25	[79.3, 86.6]	Q6_K vs Q4_K_XL	4	−1.00	0.125

99.4 % top-1 agreement. That is about half the size of the step from Q6_K_XL to Q6_K (0.0058). It is a comparison of sizes; KL divergences from a common reference do not add, and no cell applies both perturbations.

This number should not be carried to long context. It was taken at 2,048 tokens, where the cache barely matters. The literature says 4-bit KV needs per-channel key quantization to stay near-lossless [26, 27], which q4_0 lacks, and a BF16-referenced community measurement on the sibling Qwen3.6-27B over about 250K tokens reports KLD of 0.024 for q8_0 and 0.09–0.12 for q4_0 [28]: one to two orders of magnitude above my short-context figure. I could not measure divergence at depth on this host (section 3). The second battery handles the question differently: it finds the gentlest KV precision each build can hold at the full window (section 8.3) and measures task accuracy at that precision (section 10.2).

8 Context: fitting 262,144 tokens into two 16 GB budgets

8.1 What the KV cache costs on this architecture

Qwen3.8-27B is a hybrid model: of its 64 layers only every fourth uses full attention, and only those 16 layers keep a KV cache. With 4 KV heads of dimension 256, a token costs $2 \times 16 \times 4 \times 256 = 32,768$ cached elements. In llama.cpp’s block formats that is exactly **64 KiB per token at f16, 34 KiB at q8_0 and 18 KiB at q4_0**, summed over both cards. At the native 262,144 tokens the cache is 16.0, 8.5 or 4.5 GiB. The server log confirms it (llama_kv_cache: size = 4608.00 MiB (262144 cells, 16 layers) at q4_0). The remaining 48 layers carry a fixed recurrent state of 449 MiB regardless of length. MTP adds a one-layer draft cache that is always f16: 1,024 MiB at the full window. A dense 27B model with 64 attention layers would need four times as much, so the hybrid design is what makes 262K tokens thinkable on this hardware.

An earlier “VRAM model” in my own notes, fitted to measurements from the deleted

Table 10: HumanEval+, first battery, paired. 164 problems, Qwen’s official non-thinking sampling, same seed, context 32,768, **no speculation on either arm**, zero empty responses.

build	HumanEval	Wilson 95 %	HumanEval+	Wilson 95 %
Q6_K_XL	94.51 % (155)	[89.9, 97.1]	90.24 % (148)	[84.7, 93.9]
Q4_K_XL	93.90 % (154)	[89.1, 96.7]	89.63 % (147)	[84.0, 93.4]

paired, Q4_K_XL – Q6_K_XL: base -0.61 pts, 95 % score CI $[-3.80, +2.28]$, 3 discordant (1 vs 2), min. p 0.25
plus -0.61 pts, 95 % score CI $[-4.19, +2.74]$, 5 discordant (2 vs 3), min. p 0.0625

Table 11: HumanEval / HumanEval+ pass@1^H, 164 problems, greedy. Monotone in bit-width, but only the 3-bit step exceeds the ± 4.6 -point interval. With reasoning on (4,096-token budget) every build scores lower, and the builds differ mainly in how often they run out of budget and return nothing. MTP and DFlash2 rows are not comparable at a fixed build (section 9.1).

mode	build (speculation)	HE	HE+	Wilson 95 % (HE+)	empty
non-thinking	Q3_K_XL	84.1	81.7	[75.1, 86.9]	0 %
	IQ4_XS	90.2	87.8	[81.9, 92.0]	0 %
	Q5_K_XL	93.3	90.9	[85.5, 94.4]	0 %
	Q6_K_XL	93.9	91.5	[86.2, 94.9]	0 %
	Q6_K_XL, window 32K and 131K (control)	93.3	90.2	[84.7, 93.9]	0 %
thinking	IQ4_XS (MTP)	86.6	86.0	[79.8, 90.5]	12.8 %
	Q4_K_XL (MTP)	87.8	86.0	[79.8, 90.5]	12.2 %
	Q5_K_XL (MTP)	89.0	86.0	[79.8, 90.5]	11.0 %
	Q6_K_XL (MTP)	90.9	88.4	[82.6, 92.5]	7.9 %
	IQ4_XS (DFlash2)	89.0	87.8	[81.9, 92.0]	10.4 %
	Q4_K_XL (DFlash2)	90.2	87.2	[81.2, 91.5]	8.5 %
	NVFP4 on vLLM (none)	85.4	84.1	[77.8, 89.0]	12.8 %

tensor-split build, had per-GPU rates that were not per-token KV at all, and predicted ceilings that did not reproduce. The arithmetic above replaced it. It is only part of the footprint: compute buffers, the drafter and per-card placement decide the real ceiling, which has to be measured.

8.2 The default split strands memory; a manual split recovers it

Under layer split the last card also holds the output head and, with MTP, the draft context, so the default placement overloads GPU1. Figure 5 shows the effect for Q6_K. One flag moved the ceiling from 196,608 to 262,144 tokens.

Table 12 gives all four builds. Three findings.

For three of four builds the split, not the quantization, set the reachable window. Q5_K_XL and Q6_K fail at the default split and reach the native maximum once rebalanced; Q6_K_XL goes from 131,072 (its earlier default-split ceiling) to 212,992. The smallest build is the counter-example: it loads at the default. I first wrote this finding as “the ceiling is a property of the split” for the whole ladder; review found the counter-example in my own data. A published context ceiling without its split is incomplete either way.

The best split does not transfer. Q6_K wants 58,42. On Q6_K_XL that same ratio overshoots (GPU0-heavy by 2.2 GiB) and only 56,44 loads; on Q6_K, 54,46 fails while ratios on both sides of the working pair also fail. The relationship is not monotone. Re-sweep after any change of build, KV precision or speculative setting.

Balanced is not fastest. On Q5_K_XL at the full window the most balanced split (28 MiB apart) decoded at 8.50 tok/s; four other splits with 166 to 1,750 MiB of imbalance clustered at 10.4–10.8 tok/s. The outlier is a single reading and unexplained. The rule that survives: keep

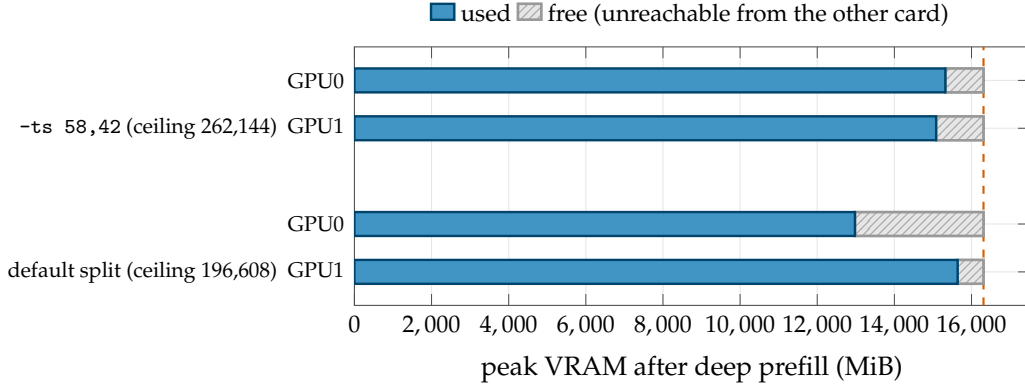


Figure 5: Q6_K with MTP and q4_0 KV, build E-A, greedy. At the engine’s default layer split the largest window that worked was 196,608 tokens: GPU1 was 671 MiB from its limit while 3.3 GiB sat idle on GPU0. Moving layers to GPU0 with `-ts 58,42` reaches the native 262,144 with both cards within 1.3 GiB of full. The dashed line is the 16,311 MiB card limit.

Table 12: Context ceilings, first battery: MTP $n=2$, q4_0 KV, layer split, `-fit off`, official non-thinking sampling, 192-token generation after a $\geq 94.7\%$ prefill. Failures at 262,144 are single attempts except where noted; all are compute-buffer out-of-memory errors read from the saved server log.

build	ceiling	splits that load	splits that failed	engine default	GPU0 / GPU1 MiB
Q6_K_XL	212,992 ^a	56,44	(at 196K) default, 52,48, 54,46, 58,42	not tried at ceiling	15,656 / 15,414
Q6_K	262,144	58,42; 56,44	default, 54,46, 60,40, 62,38	fails	15,322 / 15,082
Q5_K_XL	262,144	54,46 to 62,38 (five)	default	fails	14,660 / 15,402
Q4_K_XL	262,144	default; 56,44	—	loads	12,116 / 15,094

^a Bracketed: 229,376 failed twice. The other three reach the model’s maximum, so no rung exists above them.

the fastest split that loads, not the most balanced one. Otherwise the split has little effect on speed: with the generated text held fixed, it moved decode by at most 5.7 % in 11 of 12 groups.

8.3 The full map: context $\times$ KV precision $\times$ drafter

Table 13 is the configuration space the rest of the second battery draws from. Two consequences. The 4.1 GiB saved by Q4_K_XL over Q6_K buys a whole step of KV precision at the same window: Q4_K_XL holds the native 262K with an 8-bit cache, Q6_K cannot. Given section 7.6, that is a real trade: weight fidelity against cache fidelity. And the two drafters reach the same window except with an unquantized cache, where DFlash2’s separate 1.1 GB model costs one rung. Two passing cells peaked above 15,700 MiB on a card, past what I had treated as a practical limit; the soak in section 8.5 is what tests whether such cells are stable.

8.4 Three ways a long-context server fails

A ceiling search has to distinguish at least three failure modes, all seen on this host:

1. **Out of memory at load.** The clean case: the server log names the buffer and the device.
2. **Hang during initialisation.** Q4_K_XL at 229K and 262K on the default split, in the ceiling runs that preceded the first battery: no error in ten minutes, a 572-byte log. Not an OOM, and not to be labelled one.
3. **Death at the first speculative draft.** Q4_K_XL with DFlash2 and f16 KV at 196,608 tokens: the server loaded, prefilled all 184,807 tokens at 612 tok/s, then aborted inside the drafter on a cuBLAS workspace allocation with 15.8 GiB in use on each card. Twice. The client

Table 13: Largest window that loads, reports the requested size, prefills $\geq 94\%$ and generates 512 tokens. Second battery, upstream build E-C, layer split at each cell’s best ratio, 71 launch attempts; every ceiling below the native window is bracketed by a rung above it that failed twice. DFlash2 cells place the drafter on GPU1 (-devd CUDA1). Bold: the full native window.

build	drafter	q4_0 KV (18 KiB/tok)	q8_0 KV (34 KiB/tok)	f16 KV (64 KiB/tok)
Q6_K	MTP	262,144	196,608	131,072
Q6_K	DFlash2	262,144	196,608	98,304
Q4_K_XL	MTP	262,144	262,144	196,608
Q4_K_XL	DFlash2	262,144	262,144	163,840

saw only a dropped connection. A search that checks loading alone, or reads a disconnect as a client fault, records this rung as usable.

This is why the ceiling rule in [section 6](#) requires a generation after the prefill.

8.5 Host RAM, not VRAM, caused the only outage

After the first battery the machine went into daily use serving Q6_K with MTP at 262K. During a long agent session the server’s *host* memory grew from 2.9 to 10.0 GiB while VRAM stayed flat. Swap filled and prefill fell from 839 to 0.15 tok/s. The cause: llama.cpp keeps context checkpoints (snapshots that allow a conversation to roll back without reprocessing, needed because recurrent layers cannot be rewound [58]) in host RAM, and with MTP each one includes the draft cache for the whole sequence. Measured from a verbose trace: **150 MiB plus about 3.9 KiB per token, 1.14 GiB each at 252K tokens**. I had raised the checkpoint count from 4 to 32 because an A/B test showed slightly faster decoding at identical VRAM. That was true and beside the point: no VRAM measurement, and no ceiling test made of one prefill and one generation, can see this.

The fix was -ctxcp 4, a container memory limit, and a multi-turn soak test as a gate for any serving change: 44 turns growing to 252K tokens, watching host memory, swap and prefix-cache reuse. Under that soak ([figure 6](#)) MTP peaks at about 5.4 GiB of host memory and DFlash2 at about 1.75 GiB on both builds. On a 14 GiB host serving a 262K window, the drafter is a host-memory decision before it is a speed decision. Each additional concurrent slot would carry its own checkpoints; that was not measured.

The native 262K window fits on 2×16 GB, with a 4-bit cache for Q6_K and an 8-bit cache for Q4_K_XL. It takes a hand-tuned layer split that must be re-swept per configuration, a ceiling test that generates after a deep prefill, and a soak test that watches host RAM.

9 Speculative decoding: MTP versus DFlash2

Two drafters were available for this model. **MTP** is a prediction head trained with the model and stored in the same GGUF file; it costs no extra weights but keeps its own one-layer f16 draft cache. **DFlash2** is a separate 1.1 GB block-diffusion drafter that proposes a block of up to 7 tokens in one parallel pass, conditioned on hidden states tapped from five target layers [34, 59]. Without speculation this machine decodes a 27B model at 12–23 tok/s into a shallow cache, depending on build and engine, and at 3.5 tok/s at 246K tokens, so a drafter is not optional for interactive use.

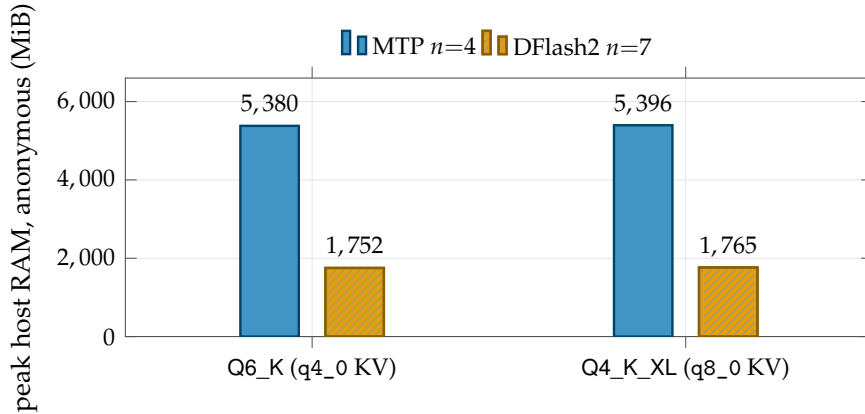


Figure 6: Peak host memory of the server process over a 44-turn conversation grown to about 252K tokens (one soak per configuration, `-ctxcp 4`, one slot). The drafter, not the build, sets the cost: about 3.6 GiB between drafters, about 16 MiB between builds. All four passed.

9.1 Speculation is a different decode path, not a free speed knob

For nine days my notes said: speculative decoding is lossless under greedy decoding, so it changes speed only, and no accuracy run needs a speculative arm. Perplexity runs for speculative configurations were skipped on that basis. The first experiment that tested the premise refuted it.

Table 14: with MTP on, 33 of 164 completions differ from the plain-decoding output, at temperature 0 with a fixed seed. A repeat a day later reproduced *both* arms byte for byte. So this is not run-to-run noise. The engine is deterministic, and speculation deterministically takes a different path for about one problem in five. Depth 2 and depth 4 each change 33 problems but share only 25 of them.

What this does and does not mean. Task accuracy did not move: pass@1 differs by at most 1.3 points across the four rows, far inside ± 4.6 . The changed completions are different programs, not worse ones. “One in five” is also a fact about HumanEval+’s output lengths: the chance of divergence rises with completion length, from 2.4 % of the shortest quartile of problems to about 50 % of the longest, a hazard near one divergence per 950 generated tokens that grows with position. That pattern fits accumulating numerical differences better than a broken acceptance rule.

The mechanism is not established. The leading explanation is batch-shape numerics: verifying $n+1$ tokens in one pass runs different GPU kernels from decoding one token, the logits differ in their last bits, and a near-tie flips. Kernels of this kind are deterministic run to run yet vary with batch shape [37], which is exactly what byte-identical repeats plus cross-configuration differences look like. At one point I read the determinism result as ruling this explanation out; it does not, and that reading is withdrawn. Others report the same behaviour on llama.cpp with quantized targets [39], and vLLM states its guarantee as lossless “up to hardware numerics” [38]. I did not compare logits at a divergence point, so this remains the leading hypothesis. All of it is greedy-only; sampling at temperature above zero was not tested.

Treat the speculative setting as part of the configuration under test. A reproducible pipeline must pin it, not only the seed, and a quantization comparison should run without speculation on both arms or with the identical setting on both.

Table 14: Speculation at context 32,768, first battery. Q6_K, greedy (temperature 0, top- p 1, fixed seed), 1,024-token limit, all 164 HumanEval+ problems, zero empty responses. “Byte-exact” compares each completion with the no-speculation baseline. The DFlash2 row ran on a different engine build (E-B) and an older drafter file, so its equivalence and speed are not comparable with the MTP rows.

configuration	tok/s (median)	speed-up	acceptance	byte-exact	HE %	HE+ %
no speculation	18.46	1.00×	—	164/164 ^a	94.5	91.5
MTP $n=2$	37.44	2.03×	0.954	131/164 ^a	93.9	90.2
MTP $n=4$	47.03	2.55×	0.892	131/164	93.9	90.9
DFlash2 $n=4$ (E-B)	51.78	2.80×	0.917	132/164	93.3	90.2

^a Re-run unchanged a day later: all 164 completions byte-identical to the first run, in both arms.

9.2 First-battery results, and why they cannot rank the drafters

The DFlash2 row of [table 14](#) was the fastest at 32K. At depth, the first battery recorded DFlash2 failing to load above 163,840 tokens and collapsing to 8 tok/s there. I reported that as “fastest when shallow, dominated when deep” and then had to take most of it back:

- the at-depth speed was timed over a 17-token generation (the probe sent a corpus to the chat endpoint with no instruction; the model acknowledged it and stopped). The same defect voided MTP’s first at-depth figures and every historical acceptance of exactly 1.000;
- the load failures at 213K and 262K were the drafter’s 738 MiB weight buffer failing to fit on GPU0 under a split tuned for MTP: a placement problem, not a DFlash2 limit;
- the arm ran on a different engine build, with a drafter file that predated an upstream metadata fix, at draft depth 4 where the drafter’s block size of 8 makes 7 the design point.

Between the batteries DFlash2 support was merged into upstream llama.cpp [36], which removed the engine confound and made a fair comparison possible.

9.3 Same build, both drafters, to 246K tokens

The second battery ran both drafters on build E-C with the current drafter file, each at several draft depths, each as a separate server launch. (On this build the draft depth cannot be changed per request: the per-request fields are compiled out, while the properties endpoint still echoes a value, so a per-request sweep would silently measure one depth.) Every cell was filled with the same real code to 32K, 131K or 246K tokens, then asked for 1,024-token continuations.

[Figure 7](#) gives the result. At 246,415 filled tokens DFlash2 at $n=7$ decodes at 25.7 against MTP’s best 17.3 tok/s on Q6_K (+49 %) and 27.7 against 17.0 on Q4_K_XL (+62 %). At 131K the margins are +45 % and +54 %; at 32K, +64 % and +45 %. MTP’s best depth is 4 in two of those cells and 3 in the other four ([section C](#)). The ranges of DFlash2 at $n=7$ and the best MTP depth overlap in one cell only (Q6_K at 246K, by 0.15 tok/s). Three details matter for anyone reproducing this:

The advantage appears at DFlash2’s design depth and not at a shared shallow depth. At a shared depth of 3 the two drafters are within noise of each other everywhere. MTP gains little from going deeper (its depth 2, 3 and 4 medians are within 10–22 % of each other, and depth 4 does not beat depth 3 consistently), while DFlash2 gains 40–70 % from depth 3 to 7 because its block is drafted in one parallel pass.

DFlash2’s accepted length grows with depth of context. Mean tokens accepted per verification pass rose from 4.4 to 4.6 on Q6_K and from 3.5 to 4.6 on Q4_K_XL between 32K and 246K. The “acceptance collapse at depth” in my early notes was the short-generation artifact

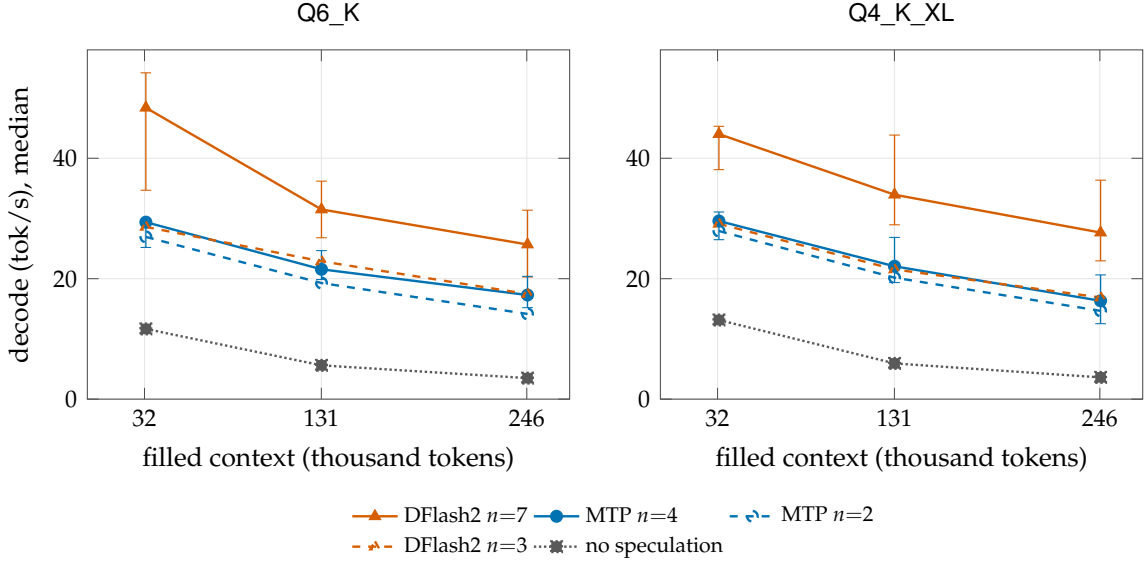


Figure 7: Decode rate against filled context, both drafters on one upstream build (E-C). Greedy, identical code prompts, q4_0 KV, 1,024-token continuations, 8 per cell (4 at 32K); markers are medians and bars the full range across continuations. At its design depth ($n=7$) DFlash2 leads at every depth on both builds. At a shared shallow depth ($n=3$, dashed) the two drafters are indistinguishable. Without speculation decode falls to 3.5 tok/s at 246K.

described above. Raw acceptance ratios are not comparable between drafters (they divide by different depths); report accepted length and tokens per second.

The decode speed-up from speculation grew with filled context. Against no speculation the best drafter gives $3.4\text{--}4.2\times$ at 32K, $5.6\text{--}5.7\times$ at 131K and $7.4\text{--}7.7\times$ at 246K, because each verification pass amortises an ever more expensive attention read over several tokens. DFlash2’s prefill is also 34–52 % faster at depth (201 against 135 tok/s at 246K on Q6_K), since MTP prefills its draft cache as well.

The pre-registered selection rule compared decode on thinking-mode requests of about 100K input tokens (three per configuration; median [min, max]). It chose DFlash2 on both builds: 20.4 [19.8, 22.7] against 16.3 [16.1, 16.6] tok/s on Q6_K, cleanly; 29.8 [23.8, 51.3] against 17.4 [15.9, 22.1] on Q4_K_XL, where the ranges do not overlap but one range spans a factor of two over three requests. The matched-depth probe above is the stronger evidence for the same conclusion. The accuracy batteries in [section 10.2](#) then served as a long-generation speed check over 756 real answers: DFlash2 decoded at a median 34.6–48.9 tok/s against MTP’s 23.7–28.2, at lower energy per request (28–41 kJ against 45–78 kJ).

9.4 Draft depth and the acceptance trap

The first battery swept MTP depth $\{2, 4, 8\}$ at matched depths of 131K and 196K on all four builds, three 512-token generations per cell, to learn the best depth per build. It could not answer. Repetition-to-repetition decode spread reached 166 %, and acceptance within one cell ranged by up to 0.63. Two lessons survive.

Clustered events give falsely tight intervals. Pooled over 1,000–4,000 draft events, a Wilson interval on acceptance is ± 0.02 . The events inside one generation are strongly correlated; the effective sample is the three generations, and their spread is ± 0.3 .

“Acceptance falls with draft depth” is nearly a tautology. It fell in 10 of 11 adjacent pairs (one-sided sign test $p = 0.006$, which fails a Holm correction at its rank). For any per-token match

probability $q < 1$, the ratio accepted/drafted must fall as depth grows. Inverting the stop-at-first-mismatch formula gives $q \approx 0.68\text{--}0.95$, roughly constant across depth within a cell. q carries the information; the ratio does not. At the full 262K window, MTP $n=8$ failed to load at all: its draft context no longer fits.

9.5 Where decode variance comes from

Early speed tables on this host showed 17–41 % spread between identical repetitions, which I first called unexplained noise. Two reviewers noted that decode correlates with acceptance at r^2 of 0.83–0.99. That correlation is an identity, not a finding: llama.cpp’s speculative decode rate is exactly $(1 + n \cdot \text{acceptance}) / t_{\text{pass}}$, since each target pass emits one guaranteed token plus the accepted drafts. Reconstructing generated length from pass and acceptance counts recovers 190 to 191 of 192 tokens in all 47 cells. What is informative is the residual. Dividing throughput by tokens-per-pass, with nothing fitted, collapses within-configuration spread from 16.7–40.7 % to 1.3–13.4 %; 98.7 % of within-cell variance is tokens-per-pass and 0.9 % is pass time. The “noise” was sampling variance of the generated text: those cells ran at temperature 0.7, so each repetition wrote a different continuation with a different acceptance. Pass time itself is well behaved, linear in filled depth at about 0.45 μs per token. **For controlled comparisons, run speed probes greedy with fixed prompts**, and report sampled workload throughput separately, since real acceptance depends on the text being generated. The second battery did both (this probe, and the 756 sampled answers of [section 10.2](#)). Its medians and ranges describe the continuations that were run within one launch per cell; they are not a significance test.

10 Accuracy versus quantization versus context depth

Does a 4-bit build lose more than a 6-bit build as the context fills? Divergence cannot be measured at depth on this host ([section 3](#)), and comparing free-running generations fails for a structural reason: I tried it, and greedy trajectories from different builds fork within the first 2 to 126 characters, after which every pair of builds is “completely different” (normalised edit distance 0.61–0.79) regardless of how close they are on the ladder. Adjacent builds measured further apart than the extremes. Distance between quantizations has to be measured on distributions at a fixed context, not on emitted text. That leaves task instruments: RULER in the first battery, AA-LCR and GPQA in the second.

10.1 RULER retrieval to 131K, and a headline withdrawn

Single-needle retrieval is saturated and unaffected. Both ends of the ladder score 100 % at 8K, 32K and 131K tokens, and every response finished its reasoning inside the budget, so these are real ceilings, not artifacts of lenient string matching ([table 15](#)).

The multi-key result is a budget effect. With four keys planted and one queried, Q6_K_XL scored 89/100 and Q4_K_XL 79/100 at 131K tokens. All ten discordant items favoured the larger build; $p = 0.002$; recovery 88.8 %, right inside the band Red Hat publishes for 4-bit weights at 128K [\[24\]](#). I wrote it up as the study’s long-context headline. It took 9.4 GPU-hours and the analysis script was committed before the second arm finished.

It does not measure retrieval. Re-reading the saved predictions: *not one response in either arm finished reasoning and then gave a wrong answer*. Every failure is a truncation at 128 tokens. On the 55 items where neither build hit the budget, both score 55 of 55. What differs is how often the reasoning block closed in time: 77 against 60, itself significant ($p = 0.0015$). The 4-bit build “thought” longer on the harder task. The closure audit also shows why depth is not the cause: at

Table 15: RULER, first battery: RULER’s own generators and `string_match_all` metric, greedy, no speculation, one split for both builds, q4_0 KV. The harness used the raw completion endpoint with a **128-token output budget and reasoning left on**. “Closed” means the output contains the end-of-reasoning tag. Prompts reached 98–100 % of the target length in every cell.

task	context	build	n	score %	closed	closed and wrong
single needle	8K / 32K / 131K	Q6_K_XL	25 / 25 / 12	100 / 100 / 100	all	0
single needle	8K / 32K / 131K	Q4_K_XL	25 / 25 / 12	100 / 100 / 100	all	0
multi-key (4 keys)	131,072	Q6_K_XL	100	89.0	77	0
multi-key (4 keys)	131,072	Q4_K_XL	100	79.0	60	0
multi-key, paired: score discordance 10 vs 0, exact McNemar $p = 0.002$, -10.0 pts $[-15.9, -4.1]$						
closure discordance 22 vs 5, $p = 0.0015$, -17.0 pts $[-26.6, -7.4]$						
on the 55 items where <i>neither</i> build hit the budget: 55/55 vs 55/55, zero discordance						

the same 131K tokens, same builds, same budget, single-needle closes 12 of 12 and multi-key closes 77 and 60 of 100. Task difficulty drives the reasoning length.

So the retrieval question at 131K on the multi-key task is *unanswered*, not answered in the negative, and the comparison with the Red Hat band is void. What remains is a smaller finding of its own: a more heavily quantized build spent measurably more tokens reasoning about the same questions. The same budget mechanism explains the thinking-mode HumanEval+ rows in [table 11](#), and a harder RULER task (variable tracking) was excluded from the battery for the same defect in its extreme form (0 of 25 closed). A score alone cannot tell a real ceiling from a truncation; the closure column can.

10.2 AA-LCR and GPQA Diamond: long-document reasoning with a short-context control

The second battery avoided those traps. It used the chat endpoint with the model’s documented reasoning control at its highest effort (`xhigh`), Qwen’s official thinking sampling, and an output budget large enough that truncation was rare (1 of 756 answers). Two instruments ran under identical settings:

AA-LCR [25]: 100 questions, each over a set of real documents totalling about 100K tokens, requiring reasoning across them. Served at the full 262K window with each build’s gentlest workable cache: q4_0 for Q6_K, q8_0 for Q4_K_XL. Answers took 2 to 60 minutes each. Correctness is judged by an LLM equality checker using the benchmark’s published prompt.

GPQA Diamond [47]: 198 graduate-level science questions of about 200 tokens, scored by answer extraction. This is the short-context control: if a drafter or build broke reasoning in general, it would show here. Single answers reached 94,000 generated tokens. With short prompts the cache could be gentler still: q8_0 for Q6_K and f16 for Q4_K_XL.

Design: all four build $\times$ drafter configurations ran a paired subset (30 AA-LCR questions, 50 GPQA items); then the faster drafter (DFlash2, [section 9.3](#)) ran the full sets on both builds. A full pass with MTP was not run, by design, to stay inside a 117-hour GPU budget. Build and cache precision change together between arms, so the comparison is between *configurations*, never between quantizations alone.

Result. [Figure 8](#): every interval overlaps every other within each instrument. On the full AA-LCR pass Q4_K_XL scores 87/100 and Q6_K 83/100 (resolved); on full GPQA, 175/198 and 178/198. The nominal order flips between the two instruments, as it did between HumanEval+ and SWE-bench in the first period. On the paired subsets the drafters do not separate either (AA-LCR 22–26 of 30; GPQA 45–48 of 50). Speculation at depth 7 on long reasoning chains did

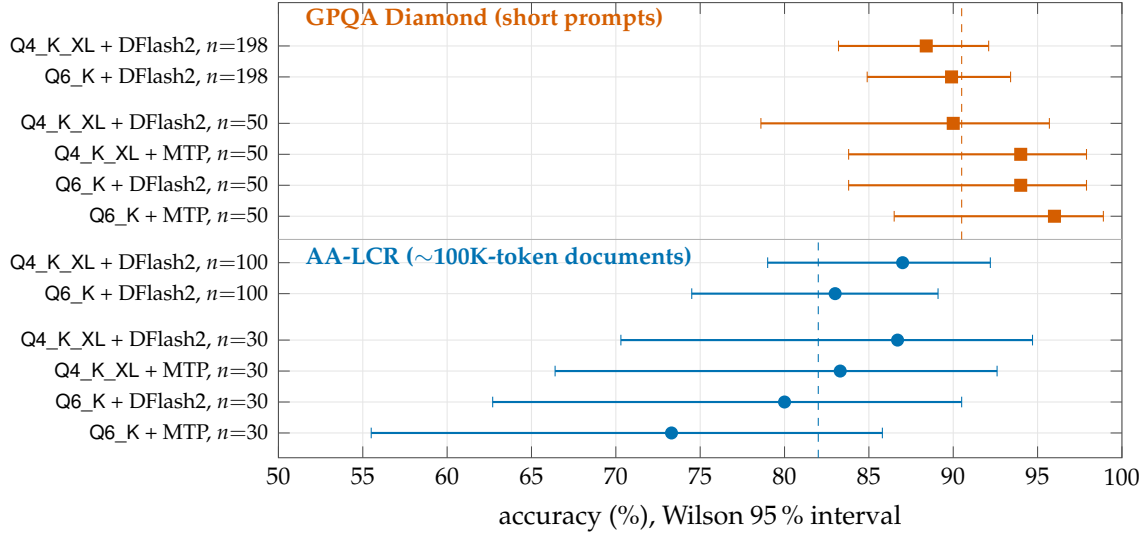


Figure 8: No build and no drafter separates from any other on either instrument. Second battery, build E-C, reasoning effort xhigh, official thinking sampling. AA-LCR rows are the three-judge resolved scores; the $n=30$ DFlash2 rows are scored on the paired-subset items, where the resolved and first-judge scores coincide. Dashed lines are Artificial Analysis’ figures for the same model served through the vendor’s API (0.820, 0.905), shown for orientation: different serving stack, precision and, for AA-LCR, judge. Build and cache precision change together between rows: the comparison is between configurations, never between quantizations alone.

Table 16: AA-LCR judging. The same 260 answers were scored by two independent LLM judges using the benchmark’s equality-checker prompt; a third judge ruled on the 11 disagreements. Identifiers are as recorded in the artifacts.

judge	Q4_K_XL+DFlash2 (/100)	Q4_K_XL+MTP (/30)	Q6_K+DFlash2 (/100)	Q6_K+MTP (/30)	total (/260)
judge 1 (muse-spark, effort xhigh)	88	25	84	22	219
judge 2 (gpt-5.6-luna, effort medium)	87	25	87	23	222
resolved (third judge on disagreements)	87	25	83	22	217
Cohen’s κ over all 260: judge 1 $\times$ 2: 0.836 (agreement 249/260); 1 $\times$ 3: 0.972; 2 $\times$ 3: 0.869					

not cost accuracy that these instruments can see, at $n = 30$ to 198.

This is the second battery’s version of the report’s thesis: 756 long answers, about 69 GPU-hours, and a 4-bit build with an 8-bit cache is indistinguishable from a 6-bit build with a 4-bit cache at the full window. Divergence ordered the same two weight builds in every one of 32 windows, in forty minutes. Both statements are true. They answer different questions, and only the first is about outcomes a user would notice.

Judging. An LLM-judged benchmark carries judge variance, and a two-of-three majority is a consensus label, not ground truth. The Wilson intervals in figure 8 cover neither judge error nor the clustering of questions within shared document sets. Table 16: the two full judges differ by 3 items overall and by at most 3 on any arm; the resolved score sits in a 5-item band (217 to 222). A hand audit of the disagreements found no pipeline defect. It did find that the misses are not scattered: a small set of questions recurs across arms where the official key is arguable (a rounding convention, a conditional answer scored against an unconditional key, an entity list with one debatable member). Four to six items decide every subset contrast, and several of those are key-ambiguous. AA-LCR scores should always name the judge and its settings.

Against the public baseline. Artificial Analysis reports 0.820 on AA-LCR and 0.905 on GPQA Diamond for this model at xhigh effort through Alibaba’s API [60]. The local figures (83–87 % and 88–90 %) are in the same range. The comparison is loose in several ways: a hosted model presumably at full precision against local 4- to 6-bit builds with quantized caches; an undisclosed sampling temperature on their side; for AA-LCR a different judge. It supports “local quantized serving is in the neighbourhood of the hosted model on these two instruments”, and no more.

10.3 What depth does to speed

Depth costs speed far more visibly than it costs accuracy. Without speculation, decode falls from 11.7–13.1 tok/s at 32K filled tokens to 3.5–3.6 at 246K (figure 7): attention reads the whole cache for every token, and under layer split the card that owns a layer does all of that work. Prefill falls from about 490–530 tok/s to 215–230. With DFlash2 the same 246K context still decodes at 26–28 tok/s. A speed table measured into an empty cache (which is what most published tables are, including my own early ones at 57.5 tok/s) overstates full-window speed by 3–5×. Any speed figure should state how full the cache was.

Up to 131K tokens, single-needle retrieval is perfect at 4 and 6 bits. At about 100K tokens of real documents, reasoning accuracy is 83–87 % on the two full passes and 73–87 % on the 30-question subsets, for both builds and both drafters, with overlapping intervals and a nominal order that flips between instruments. No accuracy instrument in this study shows a quantization × depth interaction; none had the power to exclude a small one. Output budget, not retrieval, produced the one “significant” long-context difference.

11 Agentic behaviour: SWE-bench Verified and step counts

The target workload is a coding agent, so the exploratory period spent four days on SWE-bench Verified with mini-SWE-agent: 50 instances per build, non-thinking, greedy, 250-step limit. All rows here are historical^H.

The most expensive benchmark ranks the ladder backwards, and that means nothing. Table 17 puts the smallest build first. The three builds span 2.1 points; one instance is worth 2 points; one build’s interval alone spans 24. These 50-instance runs do not resolve an ordering among these builds; with every attempted instance counted, two builds tie and the third is one instance behind. The excluded instances were not separated into infrastructure failures and model failures at the time (for Q6_K the one exclusion was an empty patch, which is a model failure), so the right-hand column is the safer one to quote. What the table does support: all three builds in the 4- to 6-bit range resolve about three quarters of these instances under this scaffold.

Getting even these numbers right took three generations of totals. Scoring first ran on an ARM64 laptop, and three SWE-bench evaluation images have no ARM64 variant, so specific instances were silently counted as failures for specific builds. An aggregator then read only the first of seven concatenated batch reports and published 8 resolved where the true figure was 37. Both were found by audit and repaired from per-instance records without re-running anything.

Below the ladder, a cliff that no cheap instrument saw. The 3-bit build Q3_K_XL was the fastest configuration of the whole study (116.9 tok/s into an empty cache with MTP depth 8) and scored 84.1 / 81.7 on HumanEval / HumanEval+ with no empty responses: lower than its neighbours, not alarming. Given agentic tasks, it hit the 250-step limit on **6 of 6** instances, never converging once. Q6_K on the same six instances converged on all six (mean 45 steps, median 37). I stopped the 3-bit run there, since the remaining 44 instances would have cost about 88 GPU-hours to confirm a 6-to-0 split. This is a qualitative result: $n = 6$, one scaffold, one seed, a

Table 17: SWE-bench Verified^H, 50 instances per build. “Scored” excludes instances that produced no evaluable patch; the last column counts every such instance as a failure, which is the conservative reading and puts all builds on the same 50 instances. This is *Verified*, not SWE-bench Pro (on which Qwen reports 61.7 for this model); the two are not comparable.

build	resolved / scored	rate	Wilson 95 %	resolved / 50 attempted
IQ4_XS	38 / 49	77.6 %	[64.1, 87.0]	76.0 %
Q5_K_XL	38 / 50	76.0 %	[62.6, 85.7]	76.0 %
Q6_K	37 / 49	75.5 %	[61.9, 85.4]	74.0 %

build that is not part of the main ladder and whose file is gone. It does not say where between 3 and 4 bits the failure begins. It is still the sharpest illustration of the report’s theme: on these six instances the 3-bit build exhausted its step budget every time, after single-shot scores that gave no warning of it.

Step counts are not a quality proxy in either direction. Among builds that converge, mean steps on three shared instances were 23 for IQ4_XS and 29 for Q5_K_XL, against about 45 for the 6-bit build on a different instance set. Per-instance counts overlap and one instance reverses the order, so nothing is separated at $n = 3$. The instance sets do not match across these runs, so the counts cannot be read as an efficiency ordering of the ladder. Since step count multiplies token cost much more than tokens-per-second does, agentic efficiency deserves a properly powered study; this was not one.

12 Speed, energy and what the instruments cost

12.1 A smaller build is not a faster build at the full window

Table 18: Decode at the full 262,144 window (prefill to 94.8 %), first battery, MTP $n=2$, q4_0 KV, official non-thinking sampling, 192-token generations. Groups match on build, context, split and checkpoint setting. Spread is (max – min)/median; the adjusted column divides each reading by its tokens-per-pass (section 9.5).

build	split	n	readings (tok/s)	median	spread	spread, adjusted
Q5_K_XL	54,46	3	10.82, 12.70, 12.94	12.70	16.7 %	2.9 %
Q6_K	58,42	3	10.75, 11.90, 14.16	11.90	28.6 %	9.7 %
Q6_K	56,44	3	10.22, 11.71, 14.99	11.71	40.7 %	11.2 %
Q4_K_XL	56,44	1	12.10	—	n/a	—

At the full window the three builds that reach it decode within 7 % of each other (table 18), while repetitions of one configuration spread by up to 41 %. The common expectation that a smaller quantization buys speed does not hold here: at depth the attention read over the cache dominates each pass, and it is the same size for every build. The second battery agrees under cleaner conditions: with MTP at about 100K tokens Q6_K and Q4_K_XL decode at 16.3 and 17.4 tok/s; per-pass time at 262K orders the builds by size (184, 191, 194 ms) with overlapping ranges. The smaller build earns its place through memory (a gentler cache at the same window, section 8.3), not through throughput. That table went through three versions in my notes (32.9 %, 46.7 %, 40.7 % for the same spread) before one was computed with a stated estimator over matched cells; see section 14.

12.2 Power and energy

Over a representative 12-hour window of the first battery (26 % GPU-busy; 43,182 one-second samples), the two GPUs drew 1.08 kWh and the CPU package 0.17 kWh. A modelled system total, adding a fixed platform allowance, is 1.79 kWh: 149 W mean, 80 W median, 409 W peak, 334 W mean under GPU load. GPU0 peaked at 90 °C and GPU1 at 76 °C. The manual splits put more layers on GPU0, which is the likely cause, but airflow between the slots was not controlled, so this is an association. A split chosen for context also selects a thermal operating point.

Per-token energy exists only for decode into an empty cache^H (table 4, and in the same series 3.7 J/token for the smallest build with DFlash2 up to 11.8 for Q6_K_XL without speculation): speculation cuts energy per token by 2–2.5× at every build, because the cards draw similar power for fewer seconds. At depth, the second battery recorded GPU energy per *request*: a median 37–41 kJ per AA-LCR answer with DFlash2 against 45–51 kJ with MTP, and 28–35 kJ against 66–78 kJ on GPQA’s long reasoning chains. These include prefix-cache effects and whatever else the host was doing, and 1 Hz sampling cannot resolve decode bursts, so they are per-request figures only. A controlled joules-per-token curve against depth was planned and cancelled.

12.3 What each instrument bought

Figure 9 is the economics of the thesis. The comparison is loose (fixed token budgets against fixed problem counts, single runs), and it answers different questions: divergence says *how far the distribution moved*, a benchmark says *whether outcomes changed*. For choosing between neighbouring quantizations of one model, the first is cheap and decisive, and the second is expensive and, at public benchmark sizes, silent. An earlier draft of mine claimed “20 minutes against 20 hours”; the 20 minutes described one cell and the 20 hours had no source. The figure shows the measured totals.

12.4 Multiple comparisons

The first battery ran 19 formal tests: nine divergence comparisons (three build pairs in three domains), six HellaSwag pairs, two HumanEval+ pairs, the RULER closure test and the draft-depth sign test. All were two-sided except the sign test, and none was pre-registered except the RULER analysis, so the family is exploratory. Under a Holm correction, which tolerates dependence between tests, the nine divergence comparisons pass whether computed from the tool’s token-level errors or from the paired window-level analysis of table 6 (smallest paired t is 4.1 on 31 degrees of freedom, $p \approx 3 \times 10^{-4}$, against a Holm threshold near 0.005), and so does the RULER closure difference. The draft-depth sign test ($p = 0.0059$) fails Holm at its rank; its eleven comparisons also share cells, so they are not independent, and I treat it as directional only. No task-benchmark comparison passes. Given their observed discordance, six of those eight could not have returned $p < 0.05$.

13 The resulting configuration

This section is the practical answer for this machine, chosen in the priority order accuracy, then usable context, then speed. It selects a complete serving configuration, not the better weight quantization, and it is a decision for one host and one workload (a single-user coding agent). The rest of the report does not depend on it.

```
llama-server -m Qwen3.8-27B-UD-Q4_K_XL.gguf -ngl 99 -sm layer -ts 58,42 \
```

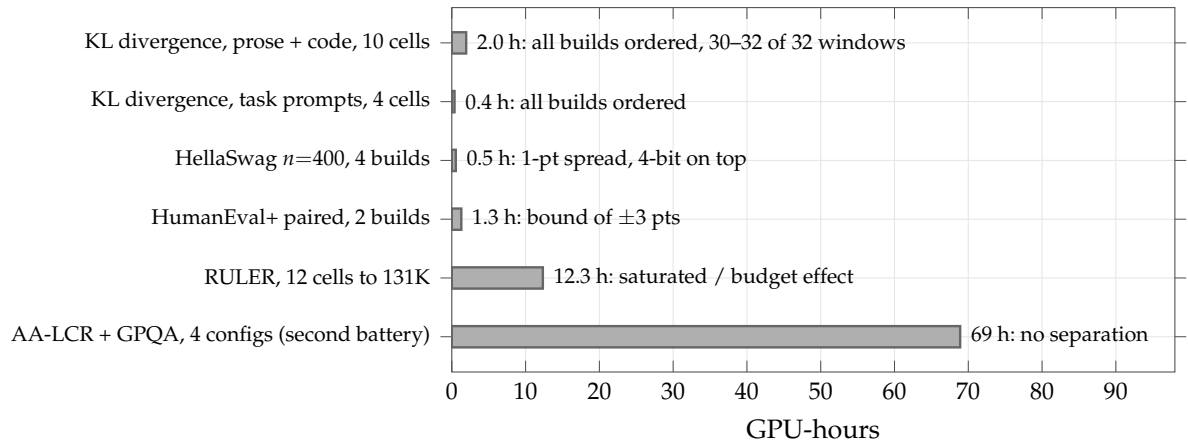


Figure 9: What a few hours of the same machine bought from each instrument. Wall-clock including model loads, single runs (the second-battery bar sums per-answer generation time: 12 h for AA-LCR, 57 h for GPQA Diamond). Divergence ranked the ladder in 2.3 GPU-hours; about 83 hours of task benchmarks across five instrument classes bounded it and ranked nothing. SWE-bench (four days, exploratory period) predates timing instrumentation and is not shown.

Table 19: The configuration that now serves on this host, and its tested fallback. Both hold the native 262,144-token window in one slot.

	serving	fallback
weights	Q4_K_XL (16.4 GiB)	Q6_K (20.5 GiB)
KV cache	q8_0	q4_0
drafter	DFlash2, draft depth 7, drafter on GPU1	built-in MTP, depth 4
engine	upstream llama.cpp b10975 (E-C)	MTP build (E-A)
split	layer, -ts 58,42	layer, -ts 58,42
decode at ~ 100 K input, thinking	29.8 tok/s [23.8, 51.3]	16.3 tok/s [16.1, 16.6]
decode at 246K filled, greedy code (q4_0 KV probe)	27.7 tok/s	17.3 tok/s
AA-LCR / GPQA Diamond	87/100 / 175/198	22/30 / 48/50 (sub-sets)
peak host RAM, 252K-token soak	1.7 GiB	5.4 GiB

```
-c 262144 -fit off -fa on -ctk q8_0 -ctv q8_0 -b 2048 -ub 512 \
-np 1 -ctxcp 4 -cram 0 \
--spec-type draft-dflash -md Qwen3.8-27B-DFlash2-Q4_K_M.gguf \
-ngld 99 --spec-draft-n-max 7 -devd CUDA1
```

How the decision went. After the first battery the choice was Q6_K with MTP and a 4-bit cache: the highest-fidelity build that reaches the full window, since speed did not separate the builds and divergence did. The second battery changed the picture on three counts. No accuracy instrument separated Q6_K with a 4-bit cache from Q4_K_XL with an 8-bit cache at the full window ([section 10.2](#)), and outside evidence makes the 4-bit cache the riskier unknown at depth ([section 7.6](#)). DFlash2 was 45–65 % faster at every depth ([section 9.3](#)). And it used 3.6 GiB less host RAM on a 14 GiB machine whose only outage had been host RAM ([section 8.5](#)). On divergence Q4_K_XL is measurably further from the reference than Q6_K; whether a user would ever notice is exactly what the task instruments could not show. I weighed a measured speed and memory gain against an unmeasurable accuracy difference, and kept the 6-bit line one restart away.

What did not survive first contact. The first cut-over failed. The serving image (E-A) lists draft-dflash in its help text and cannot parse the drafter file: the same “expected 81, got 58” signature that had produced a silent zero a month earlier. The winner needed a different engine build, not a different flag. The second attempt, on build E-C, passed the gate that the outage had created: a 44-turn soak to 252,525 tokens under serving sampling, peak host memory 1,714 MiB, no swap growth, rollbacks reprocessing 4 tokens, and recovery from a forced kill to a healthy server at the full window.

If your hardware differs. The numbers will not transfer. The procedure should: (1) compute the KV budget from the architecture; (2) sweep the layer split for *your* build, cache precision and drafter, keeping the fastest split that loads; (3) accept a window only after a deep prefill *and* a generation; (4) prefer the gentlest cache precision that holds your window; (5) run the drafter at its design depth and measure at your real context depth with greedy fixed prompts; (6) soak-test a growing multi-turn conversation while watching host RAM before trusting any of it.

14 The study’s own defects and retractions, as method

Sixteen claims in my notes were withdrawn, superseded or narrowed before this report was written, several of them headlines. Fourteen cost no GPU time to correct, because completions, predictions, per-cell timings and server logs had been kept, not only their summaries. I report them because they are the most reusable part of the work: each is a way a careful-looking local benchmark produces a wrong number that looks fine.

Three families of defect. Almost every error in [table 20](#), and every harness bug behind them, belongs to one of three families.

(i) *An aggregate over cells that differ in an untracked variable.* Six “repetitions” that spanned four context depths. A correction of that error which then mixed two checkpoint settings in one group and quietly changed the spread estimator. A speed column holding a last repetition, a first repetition and a median side by side, which reversed a ranking. A log parser that published the first of seven concatenated batch summaries. **Rule:** every table names its estimator and *n*; group only on the full cell key; recompute headline numbers from artifacts at write-up time.

(ii) *A contract documented but not asserted.* A “prefill at least 90 % of the window” rule that lived in a docstring while a pad-builder bug delivered 79.7 %. A harness that scored a subprocess by exit status: ten divergence cells burned 1.5 GPU-hours, exited 0 and recorded nothing, because the parser expected ASCII “+/-” and the tool prints “±”; the saved logs recovered every number. A speed probe that checked the prompt depth and never checked that anything was generated, giving 52 tok/s at 131K tokens from a 17-token reply. A sweep in which every cell failed, marked done. **Rule:** assert in code that the measurement exists and has the shape you asked for; a run that measured nothing must fail loudly.

(iii) *An identifier or binding that is not unique.* A drafter launched on an engine that cannot parse it records a clean 0.000 that looks like a catastrophic model. Two cells sharing one label shared one log path, and the second run overwrote the first’s evidence inside a well-formed file. A safety check that matched process names by substring matched its own supervisor and cancelled an eleven-hour campaign in eight minutes. **Rule:** a label includes every dimension that varies; the engine image digest sits next to every number; process matching is anchored to a path.

Table 20: Selected retractions. “Before” columns must not be quoted as results.

claim before	claim after	found by
Speculative decoding is lossless under greedy; skip its accuracy runs	131/164 byte-exact; a reproducibly different decode path; task accuracy unchanged	first experiment that tested it
4-bit weights cost 10 points of retrieval at 131K ($p = 0.002$)	a reasoning-budget effect; on items where the budget did not bind, 55/55 vs 55/55	adversarial re-read of predictions
MTP $n=4$ leads $n=2$ by 46 % at 262K; DFlash2 collapses at depth; acceptance 1.000 at 250K	all timed over 17- to 55-token generations; withdrawn	implausible magnitude in a server log
The context ceiling is a property of the split, not the build	true for 3 of 4 builds; the smallest loads at the default	review
Repetition noise is 32.9 % (then 46.7 %)	40.7 % under a stated estimator over matched cells; mostly text variance, not engine noise	two reviews, then a re-check of the correction
HumanEval+ “McNemar $p = 1.0$, indistinguishable”	the test’s minimum p was 0.25; replaced by a paired interval	adversarial review
Benchmarks miss damage because they are saturated	false for 2 of 3 instruments; the general framing is prior work [3]	review
The divergence tail is something quantization does	its shape belongs to the corpus; quantization sets the level	the study’s own KV-only control
Code is perturbed “100–200×” less at the median	about 200×; the “100” was the reciprocal of a rounded cell	verification pass
Divergence took 20 minutes, benchmarks 20 hours	2.3 h against 14.3 h (first battery)	recomputed from artifacts
Three engines hit the 1.19 GiB wall	two did; llama.cpp succeeded	re-reading my own evidence table

Premises suppress measurements. The “lossless” premise did more damage than any bug. It was recorded as fact on day one, and for nine days it excused every accuracy run with a speculative arm; it also shaped the first configuration decision (“about $2\times$ decode without touching outputs”). The refutation took 1.3 GPU-hours once someone thought to run it. A claim about what an implementation preserves is an empirical claim about that implementation.

Guards are calibrated to a workload. After the host-RAM outage I added a guard that stops a benchmark when swap grows. It then stopped the GPQA battery three times while 8 GiB of RAM was free: very long generations (30–60 minutes each) let the kernel migrate idle pages to swap, which looks like the outage and is benign. Swap *activity* distinguishes the two; swap *occupancy* does not. The second battery ran with 12 GiB of swap for that reason; no answer was lost or repeated across any restart, because the guard runs after each answer is written.

Corrections are not exempt. One spread figure went through three published values because the note correcting it committed both of the defects it described. Several of the corrections above were found by independent adversarial review passes over my notes, not by me. If there is one practice to copy from this project, it is scheduling that review before writing the paper.

15 Limitations

One model, one machine, one engine family. Every number is for Qwen3.8-27B in Unsloth Dynamic GGUF builds on two RTX 5060 Ti cards with llama.cpp builds from August–September 2026. The hybrid attention layout (KV in 16 of 64 layers) makes the context arithmetic unusually

favourable; dense models of this size will not reach these windows on this memory. The split optima, ceilings and speeds are specific to this host. One operator, mostly one seed.

There is no full-precision reference. BF16 weights do not fit, so every divergence is a distance from a 6-bit build along a ladder. The reference’s own damage is unmeasured and is zero here by construction. Unsloth’s published figures suggest it is small [5]; I did not verify that. The reference build Q6_K_XL is also not the same file as the Q6_K arm carried into the second battery.

Divergence is measured at 2K tokens, on prompt tokens, on two corpora. Not at depth, not on generated text, not on other programming languages, natural languages or tool-calling formats. The calibration data of these builds is not published in detail, so a bias toward Wikipedia-like text cannot be excluded. The tool’s standard errors assume independent tokens (section 12.4).

Long-context task accuracy is thinly measured. RULER’s retrieval question at 131K on the multi-key task is open (section 10.1); the fix (a generous output budget, reasoning off) was identified and not run. AA-LCR covers about 100K tokens, not 262K, with $n = 100$ and a full pass for one drafter only. No code-editing task was evaluated at depth for any build; every agentic run used prompts near 40K tokens. Nothing in this report licenses “accuracy holds at 250K”.

The cache precision is validated only at short context. The q4_0 cost (KLD 0.003) was measured at 2K tokens on one build and one corpus. Outside evidence suggests far larger effects at depth (section 7.6). In the second battery, build and cache precision change together, so their effects cannot be separated.

Task benchmarks are underpowered by construction. That is a finding, and also a limit: a null here means “not separated at this n ”, never “equal”. HumanEval+ bounds the 4-bit to 6-bit difference at about 4 points; AA-LCR and GPQA bound it at roughly 10. A difference of one or two points could exist and would be invisible to everything I ran except divergence.

Sampling. Historical rows used greedy decoding, which matches neither of Qwen’s presets, so their absolute values are not comparable to any published score. The model card’s own benchmarks (LiveCodeBench v6, SWE-bench Pro, Terminal-Bench) were not run. The non-equivalence result for speculative decoding is greedy-only, and its mechanism is not established.

Speed and energy. Speed is single-stream, single-slot. Batch serving was out of scope. The first battery’s speed cells ran at temperature 0.7 and carry text-driven variance; the second battery’s are greedy with $k = 8$ per cell, which is small. There is no wall-power sensor; system power is modelled; no controlled energy-per-token curve at depth exists. Thermal asymmetry was observed, not controlled.

Cancelled and incomplete work. A breadth wave (speculation at temperature above zero, the presence-penalty setting, the draft cache precision) and the energy curve were cancelled to stay within budget. The vLLM context battery was stopped before its deep-prefill arm, so its ceiling is boot-verified only; the stable-versus-nightly discrepancy (98K against 51K) is unresolved. SGLang was attempted with a speculative worker only. Several context failures at the default split are single attempts. Six intermediate GGUF builds were never tested.

Historical rows cannot be re-measured. The engine image behind everything before 2026-08-29 is gone, with two model files. Those rows are labelled and kept apart. The evidence chain for most numbers ends in server logs that are too large to publish; the repository carries the derived artifacts and verbatim extracts. By the usual badge definitions this is “artifacts available”, not “results reproduced”.

Judged benchmarks depend on the judge. AA-LCR totals moved across a 5-item band among three judges, and a handful of questions with arguable keys decide every small-sample contrast.

Process. AI agents executed the campaign and assisted the analysis and the writing, under my direction. That speeds up work and errors alike; [section 14](#) is the record of how many of the latter were caught, and cannot say how many were not.

16 Conclusion

A 27B hybrid-attention model runs at its full 262K-token window on two 16 GB consumer cards, decodes at 26–28 tokens per second with that window nearly full, and answers long-document reasoning questions about as well as the hosted model does on public leaderboards. Getting there took a hand-swept layer split, a quantized cache, a drafter run at its design depth, and a soak test that watches host RAM. None of those appear in a typical quantization table.

The methodological result is about instruments. For choosing between neighbouring quantizations, every task benchmark I could afford (six, from a 30-minute multiple-choice run to four days of agentic repair) returned overlapping intervals, and more than once ranked the ladder backwards. KL divergence against a reference separated the same builds decisively in two hours, showed the damage to be about twice as large on code as on prose, and located it in a few percent of tokens. The two views are consistent: a perturbation confined to a thin tail rarely lands on a decisive token, so outcomes barely move. That link is consistent with the data and was not tested, and other explanations compete with it (coarse binary scoring, compensating flips, a single seed and decoding policy, possible benchmark exposure); the tail’s shape is also a property of the corpus, not of quantization. What a practitioner should take from the pair is a bound, not a ranking: going from the 6-bit reference to the 4-bit build moved HumanEval+ by -0.6 points with an interval reaching about four, and nothing visible on GPQA Diamond or AA-LCR.

Against the title’s three axes, then. *Accuracy*: quantization from 6 to 4 bits costs a divergence that is easy to measure ($3.7\times$ on code) and an outcome difference that no benchmark here could detect; speculative decoding changes about one greedy completion in five and no score. *Context*: quantization decides how precise a cache fits beside the weights at 262K tokens (8-bit for Q4_K_XL, 4-bit for Q6_K, and only 212,992 tokens for Q6_K_XL), and the layer split decides whether the window loads at all; a drafter costs little context with a quantized cache and one 32K-token rung, for the separate drafter, with an f16 cache. *Speed*: quantization buys almost none at depth (builds within 7 % at the full window), while speculation buys $3.4\text{--}4.2\times$ at 32K filled tokens and $7.4\text{--}7.7\times$ at 246K, with DFlash2 at draft depth 7 ahead of MTP by 45–65 % at every depth.

The same lesson held on the systems side. A context ceiling depended on the layer split. A perplexity gap went from 29 % to 0.8 % when the convention was matched. A “lossless” speed feature changed one completion in five. A drafter looked dominated until it was run on the right build, at the right depth, for more than 17 tokens. In each case the number was a property of the protocol until the protocol was pinned.

Three things I would ask of published local-inference numbers, including my own: state the corpus, window and scoring rule with every perplexity or divergence; state the split, cache precision and filled depth with every context or speed figure; and keep per-item records, because that is what made fourteen of sixteen corrections free.

Artifact availability

Data, harness source, the decision log, the ledger and all per-finding notes (including the superseded ones) are at <https://github.com/henrique-simoes/llm-quantization-damage> under CC BY 4.0 (data and text) and an open-source licence (code). Raw server logs and the 1 Hz power log are too large to publish; derived artifacts and verbatim extracts stand in for them. AA-LCR documents and GPQA items are not redistributed, per their licences; answer files carry item identifiers only.

Acknowledgements

This work depends on llama.cpp and its contributors, the Qwen team, Unsloth’s quantized builds, the DFlash authors and the maintainers who ported the drafter, vLLM, EvalPlus, SWE-bench and mini-SWE-agent, RULER, GPQA, and Artificial Analysis for publishing AA-LCR with its scoring prompt. No external funding was received. Errors are mine.

References

- [1] Qwen Team. Qwen3.8-Max: A new bar for coding and cowork, August 2026. URL <https://qwen.ai/blog?id=qwen3.8>.
- [2] Qwen Team. Qwen3.8-27B. Hugging Face model card, 2026. URL <https://huggingface.co/Qwen/Qwen3.8-27B>. Apache-2.0; created 2026-08-05. Native context 262,144. Official sampling — thinking: temp 1.0 / top_p 0.95 / top_k 20 / min_p 0.0 / presence 0.0; non-thinking: temp 0.7 / top_p 0.80 / top_k 20 / min_p 0.0 / presence 1.5. reasoning_effort options: xhigh | medium | low.
- [3] Abhinav Dutta, Sanjeev Krishnan, Nipun Kwatra, and Ramachandran Ramjee. Accuracy is not all you need. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. doi: 10.48550/arXiv.2407.09141. URL <https://arxiv.org/abs/2407.09141>.
- [4] Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++, 2026. URL <https://github.com/ggml-org/llama.cpp>. Engine commit d222767; container image llamacpp-mtp:latest, digest sha256:feb0231976b65b548772845652833a370f78890d94f485a1392e25fdb5944b62.
- [5] Unsloth AI. Unsloth dynamic 3.0 GGUFs. Unsloth documentation, 2026. URL <https://unsloth.ai/docs/basics/dynamic-3.0-ggufs>. Ranks Dynamic GGUFs on mean KL divergence; warns that calibrating and evaluating on the same Wikipedia-like data overfits the metric. A companion “Dynamic 2.0” page cited in earlier drafts no longer resolves.
- [6] Unsloth AI. unsloth/Qwen3.8-27B-GGUF. Hugging Face model repository, 2026. URL <https://huggingface.co/unsloth/Qwen3.8-27B-GGUF>. Source of every GGUF arm in this study. Exact file sizes and SHA-256 digests are recorded in manuscript/references/PROVENANCE.md.

- [7] Simon Layton. Introducing NVFP4 for efficient and accurate low-precision inference. NVIDIA Technical Blog, 2025. URL <https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/>. Publication date not independently confirmed; treat as mid-2025.
- [8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- [9] vLLM Project. vLLM: a high-throughput and memory-efficient inference and serving engine for LLMs, 2026. URL <https://github.com/vllm-project/vllm>. Versions used: stable 0.27.1 (image digest sha256:0a51ea5b4ae2...) and nightly 0.26.1rc1.dev1219+g46638857f (sha256:22d3b1217fda...); full digests are in the repository’s environment manifest.
- [10] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *5th International Conference on Learning Representations (ICLR)*, 2017. doi: 10.48550/arXiv.1609.07843. URL <https://arxiv.org/abs/1609.07843>. Introduces WikiText-2, the convention corpus for llama.cpp perplexity and KL-divergence measurement.
- [11] kalomaze. Perplexity / PPL, as a quantization loss benchmark, is inaccurate — KL divergence seem to be a better data point. GitHub Discussion #4110, ggml-org/llama.cpp, November 2023. URL <https://github.com/ggml-org/llama.cpp/discussions/4110>.
- [12] ggml-org llama.cpp contributors. llama.cpp perplexity tool: perplexity and KL-divergence measurement. <https://github.com/ggml-org/llama.cpp/blob/master/tools/perplexity/README.md>, 2026. URL <https://github.com/ggml-org/llama.cpp/blob/master/tools/perplexity/README.md>. Documents `-kl-divergence-base` and `-kl-divergence`. Accessed 2026-09-02.
- [13] Fireworks AI. How Fireworks evaluates quantization precisely and interpretably. Fireworks AI blog, August 2024. URL <https://fireworks.ai/blog/fireworks-quantization>. Publishes a KLD < 0.007 threshold for high-quality deployments.
- [14] oobabooga. GGUF quantization quality benchmark: Methodology. LocalBench (Substack), April 2026. URL <https://localbench.substack.com/p/gguf-benchmark-methodology>. ~250,000 tokens across 6 task categories; KLD on prompt tokens plus top-1 agreement, over top-40 log-probabilities. Argues the commonly quoted 0.01–0.03 band for Q4_K_M is an artifact of short-context Wikipedia protocols.
- [15] Uygar Kurt. Which quantization should I use? A unified evaluation of llama.cpp quantization on Llama-3.1-8B-Instruct, 2026. URL <https://arxiv.org/abs/2601.14277>.
- [16] Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations, 2024. URL <https://arxiv.org/abs/2411.00640>.
- [17] Sam Bowyer, Laurence Aitchison, and Desi R. Ivanova. Position: Don’t use the CLT in LLM evals with fewer than a few hundred datapoints. <https://arxiv.org/abs/2503.01747>, 2025. ICML 2025 position paper; arXiv:2503.01747.

- [18] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? In *First Conference on Language Modeling (COLM)*, 2024. doi: 10.48550/arXiv.2404.06654. URL <https://arxiv.org/abs/2404.06654>.
- [19] NVIDIA. RULER: reference implementation and synthetic task generators, 2024. URL <https://github.com/NVIDIA/RULER>. Task generators, official templates and the string_match_all metric reproduced verbatim and unit-checked in this study.
- [20] Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. HELMET: How to evaluate long-context language models effectively and thoroughly. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.02694.
- [21] Ali Modarressi et al. NoLiMa: Long-context evaluation beyond literal matching. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2502.05167. Full author list to be completed before publication.
- [22] Lizhe Fang, Yifei Wang, Zhaoyang Liu, Chenheng Zhang, Stefanie Jegelka, Jinyang Gao, Bolin Ding, and Yisen Wang. What is wrong with perplexity for long-context language modeling? In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. doi: 10.48550/arXiv.2410.23771. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/ebd6641c32ed633c2a3addc689d39896-Abstract-Conference.html. Proposes LongPPL; reports Pearson -0.96 with long-context benchmark performance where plain perplexity correlates poorly.
- [23] Anmol Mekala, Anirudh Atmakuru, Yixiao Song, Marzena Karpinska, and Mohit Iyyer. Does quantization affect models’ performance on long-context tasks? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025. doi: 10.48550/arXiv.2505.20276. URL <https://arxiv.org/abs/2505.20276>.
- [24] Eldar Kurtić, Mark Kurtz, Alexandre Marques, and Dan Alistarh. How well do quantized models handle long-context tasks? Red Hat Developer (originally published by Neural Magic), 2025. URL <https://developers.redhat.com/articles/2024/02/03/how-well-do-quantized-models-handle-long-context-tasks>. RULER on Llama-3.1 8B/70B at 4K–128K across FP W8A8, INT W8A8 and INT W4A16. The page’s stated date of 2024-02-03 is a site-migration artifact and precedes the model and benchmark it evaluates.
- [25] Artificial Analysis. AA-LCR: Artificial analysis long context reasoning, version 1.1. <https://huggingface.co/datasets/ArtificialAnalysis/AA-LCR>, 2026. Hugging Face dataset ArtificialAnalysis/AA-LCR, revision 9a77ef56 (2026-09-04), Apache-2.0.
- [26] Zirui Liu et al. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.02750. Full author list to be completed before publication.
- [27] Coleman Hooper et al. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2401.18079. Full author list to be completed before publication.
- [28] oobabooga (localbench). Gemma 4 and qwen 3.6 with q8_0 and q4_0 KV cache: KL divergence results. <https://localbench.substack.com/p/kv-cache-quantization-benchmark>, 2026. Published 2026-04-24.

- [29] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>.
- [30] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling, 2023. URL <https://arxiv.org/abs/2302.01318>.
- [31] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*. PMLR, 2024. URL <https://proceedings.mlr.press/v235/cai24b.html>.
- [32] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction, 2024. URL <https://arxiv.org/abs/2404.19737>.
- [33] DeepSeek-AI. DeepSeek-V3 technical report, 2024. URL <https://arxiv.org/abs/2412.19437>.
- [34] Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. PMLR, 2026. doi: 10.48550/arXiv.2602.06036. URL <https://arxiv.org/abs/2602.06036>. Authors’ implementation: <https://github.com/z-lab/dflash>.
- [35] Inco AI. DFlash 2: Keep drafting parallel. <https://inco.ai/blog/dflash2/>, 2026. Blog post, 2026-08-18.
- [36] ggml-org. spec : add DFlash2 support (local convolution + candidate selector), llama.cpp pull request #27342. <https://github.com/ggml-org/llama.cpp/pull/27342>, 2026. Merged 2026-08-27.
- [37] Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>, 2025. Blog post, September 2025.
- [38] vLLM project. Speculative decoding: lossless guarantees. https://docs.vllm.ai/en/latest/features/speculative_decoding/, 2026. Documentation, retrieved 2026-09-15.
- [39] ggml-org. Eval bug: Speculative decoding (draft-mtp / draft-dspark): greedy output diverges from vanilla on quantized targets, llama.cpp issue #25618. <https://github.com/ggml-org/llama.cpp/issues/25618>, 2026. Opened 2026-07-13; open as of 2026-09-15.
- [40] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [41] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=1qvx610Cu7>.

- [42] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019. doi: 10.48550/arXiv.1905.07830. URL <https://arxiv.org/abs/1905.07830>.
- [43] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. doi: 10.48550/arXiv.2310.06770. URL <https://arxiv.org/abs/2310.06770>.
- [44] OpenAI. Introducing SWE-bench verified. OpenAI blog, August 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>. A 500-instance human-validated subset of the SWE-bench test set. Distinct from SWE-bench Pro; scores on the two are not comparable.
- [45] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://arxiv.org/abs/2405.15793>.
- [46] SWE-agent contributors. mini-swe-agent, 2026. URL <https://github.com/SWE-agent/mini-swe-agent>. Used for the SWE-bench Verified runs. A simpler rewrite of SWE-agent; the SWE-agent paper describes a different system.
- [47] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petkova, Richard Yuanzhe Pang, Julien Dirani, Nikita Mishra, and Samuel R. Bowman. GPQA: A graduate-level google-proof Q&A benchmark. *arXiv preprint arXiv:2311.12022*, 2023. Published at COLM 2024.
- [48] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. doi: 10.48550/arXiv.2403.07974. URL <https://arxiv.org/abs/2403.07974>.
- [49] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>. Describes Terminal-Bench 2.0 (89 tasks); leaderboard at <https://www.tbench.ai/>.
- [50] Hugging Face. Perplexity of fixed-length models. Transformers documentation. URL <https://huggingface.co/docs/transformers/en/perplexity>. Retrieved 2026-09-21.
- [51] asher. mlx-kld: KL-divergence measurement for quantized MLX models. GitHub repository. URL <https://github.com/asher/mlx-kld>. Retrieved 2026-09-21.
- [52] malaiwah. KLD method notes for Qwen3.8-27B EXL3 quantizations. GitHub repository, docs/42-kld-method.md. URL <https://github.com/malaiwah/qwen38-27b-exl3>. Retrieved 2026-09-21.
- [53] Artificial Analysis. Intelligence benchmarking methodology; LLM performance benchmarking methodology. <https://artificialanalysis.ai/methodology/intelligence-benchmarking>, 2026. Retrieved 2026-09-15.

- [54] OpenAI. simple-evals. <https://github.com/openai/simple-evals>, 2024. `common.py` multiple-choice template and answer pattern.
- [55] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.
- [56] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.
- [57] B. Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1): 1–26, 1979. doi: 10.1214/aos/1176344552.
- [58] ggml-org. Context checkpoints near the end of the prompt, llama.cpp pull request #20288. <https://github.com/ggml-org/llama.cpp/pull/20288>, 2026.
- [59] Inco AI and z-lab. Qwen3.8-27B-DFlash2 model card and GGUF release (mirrored as z-lab/qwen3.8-27b-dflash2-gguf). <https://huggingface.co/incoai/Qwen3.8-27B-DFlash2>, 2026. Q4_K_M GGUF sha256 1a25c568..., revision with dflash.rope.dimension_sections.
- [60] Artificial Analysis. Qwen3.8 27B (xhigh, medium, low, non-reasoning): Intelligence, performance & price analysis. <https://artificialanalysis.ai/models/qwen3-8-27b>, 2026. Retrieved 2026-09-15; archived locally with SHA-256 checksums.

A Full divergence table

Table 21: All ten divergence cells. llama-perplexity -kl-divergence against Q6_K_XL logits, context 2,048, q4_0 KV, build E-A. Mean $\pm$ tool-reported SE (tokens treated as independent). Quantiles carry no interval. The last row perturbs only the KV cache (q4_0 against f16) on the reference weights.

build	corpus	tokens	mean KLD $\pm$ SE	top-1 %	median	p90	p95	p99	p99.9
Q6_K	prose	65,536	.003321 $\pm$.000126	97.50	.001392	.006742	.010549	.028639	.098167
Q5_K_XL	prose	65,536	.004465 $\pm$.000281	97.14	.001810	.008856	.013936	.037323	.124839
Q4_K_XL	prose	65,536	.008207 $\pm$.000340	96.22	.003509	.016333	.025546	.069578	.259651
Q6_K	code	65,536	.005829 $\pm$.000233	99.14	.000007	.003029	.016109	.142839	.623575
Q5_K_XL	code	65,536	.010285 $\pm$.000458	98.87	.000010	.004790	.026311	.250389	1.06461
Q4_K_XL	code	65,536	.021529 $\pm$.000834	98.43	.000017	.010006	.055800	.562891	2.19352
Q6_K	task	18,432	.010403 $\pm$.000448	98.05	.000197	.020278	.050149	.189885	.575163
Q5_K_XL	task	18,432	.017285 $\pm$.000703	97.34	.000345	.035139	.082368	.313987	.807942
Q4_K_XL	task	18,432	.036129 $\pm$.001534	95.89	.000685	.077069	.175076	.616163	1.74658
Q6_K_XL, KV only	code	65,536	.002955 $\pm$.000127	99.40	.000004	.001529	.008188	.067592	.322095

Separation between builds in combined standard errors: on code 8.67 (Q6_K/Q5_K_XL), 11.82 (Q5_K_XL/Q4_K_XL), 18.13 (Q6_K/Q4_K_XL); on prose 3.71, 8.48, 13.48; on task prompts 8.26, 11.17, 16.10.

Table 22: Q5_K_XL at 262,144 tokens, first battery, MTP $n=2$, q4_0 KV, prefill to 94.8 % (248,522 tokens), first repetition of each split. Balance and speed are unrelated except for one unexplained outlier.

-ts	GPU0 MiB	GPU1 MiB	imbalance	decode tok/s	outcome
default	13,540	15,070	1,530	—	compute-buffer OOM (1 attempt)
54,46	14,660	15,402	742	10.82	ok (median of 3: 12.70)
56,44	14,946	15,112	166	10.78	ok (median of 3: 12.51)
58,42	15,498	15,470	28	8.50	ok
60,40	15,260	14,084	1,176	10.67	ok
62,38	15,546	13,796	1,750	10.44	ok

Table 23: Second battery, build E-C, greedy, identical code prompts, q4_0 KV, 1,024-token continuations. Median decode tok/s [min-max] over k continuations ($k = 4$ at 32K, $k = 8$ otherwise). Each draft depth is a separate server launch.

build	drafter, depth	32,768 filled	131,072 filled	246,415 filled
Q6_K	none	11.7	5.6	3.5
Q6_K	MTP 2	26.9 [25.8–27.3]	19.3 [18.5–20.6]	14.2 [13.7–15.8]
Q6_K	MTP 3	29.5 [26.9–29.9]	21.8 [19.8–23.1]	15.9 [15.0–19.3]
Q6_K	MTP 4	29.4 [25.2–29.7]	21.6 [19.8–24.6]	17.3 [15.2–20.4]
Q6_K	DFlash2 3	28.6 [26.5–31.4]	22.9 [19.8–24.4]	17.5 [15.0–20.7]
Q6_K	DFlash2 5	30.0 [27.5–41.3]	23.9 [19.2–28.6]	20.1 [16.7–25.0]
Q6_K	DFlash2 7	48.4 [34.7–54.2]	31.5 [26.8–36.2]	25.7 [20.3–31.4]
Q4_K_XL	none	13.1	5.9	3.6
Q4_K_XL	MTP 2	27.9 [26.9–29.2]	20.2 [18.9–22.1]	14.7 [13.6–15.9]
Q4_K_XL	MTP 3	30.3 [28.0–31.7]	21.4 [20.0–25.1]	17.0 [15.2–18.4]
Q4_K_XL	MTP 4	29.6 [26.5–31.1]	22.1 [19.4–26.9]	16.3 [12.5–20.6]
Q4_K_XL	DFlash2 3	29.1 [27.3–30.5]	21.5 [20.1–26.3]	16.9 [15.0–19.8]
Q4_K_XL	DFlash2 5	30.6 [28.2–35.4]	22.5 [20.1–33.0]	21.2 [16.6–25.4]
Q4_K_XL	DFlash2 7	44.0 [38.1–45.3]	34.0 [29.0–43.8]	27.7 [23.0–36.4]

Prefill tok/s at 32K / 131K / 246K: Q6_K none 487/309/216, MTP 397/209/135, DFlash2 424/281/201;
Q4_K_XL none 533/330/228, MTP 425/218/139, DFlash2 457/297/211.

B The layer-split sweep for one build

C Matched-depth speed probe, all cells

D MTP draft depth at matched context depth (first battery)

E A checklist for local-inference measurements

Collected from the failures in [section 14](#); each line would have prevented at least one wrong number in this study.

1. Record the engine image digest, model file hash and full launch line next to every number.
2. Send an explicit sampling block with every request and assert the server’s echo of it.
3. Probe the reasoning state (on/off) with a two-token request before each battery.
4. Launch with automatic context fitting off; assert reported context equals requested context.
5. Accept a context window only after a deep prefill with real text *and* a generation of known length; read VRAM after the prefill; try a failed rung twice.
6. Re-sweep the layer split after any change of build, cache precision or drafter.

Table 24: Acceptance ratio, with the implied per-token match probability q in parentheses, from inverting $\text{acc}(n) = q(1 - q^n) / (n(1 - q))$. Each build at its own split, q4_0 KV, official non-thinking sampling, three 512-token generations per cell, identical prompt length per rung. Decode spread between repetitions reached 166 %, so no best depth per build can be read from this table.

context	build	$n=2$	$n=4$	$n=8$
131,072	Q4_K_XL	0.673 (0.76)	0.512 (0.75)	0.404 (0.79)
131,072	Q5_K_XL	0.814 (0.87)	0.493 (0.74)	0.449 (0.82)
131,072	Q6_K	0.775 (0.84)	0.558 (0.78)	0.251 (0.68)
131,072	Q6_K_XL	0.759 (0.83)	invalid (short generation)	0.332 (0.75)
196,608	Q4_K_XL	0.666 (0.76)	invalid (short generation)	0.373 (0.77)
196,608	Q5_K_XL	0.746 (0.82)	0.429 (0.69)	0.573 (0.88)
196,608	Q6_K	0.920 (0.95)	0.790 (0.91)	0.517 (0.85)
196,608	Q6_K_XL	0.704 (0.79)	0.556 (0.78)	failed to load
262,144	Q6_K	0.738 (0.81)	0.492 (0.74)	failed to load

7. For speed: greedy, fixed prompts, a stated filled depth, and an assertion that the generation reached its requested length. Report accepted length per pass, not acceptance ratio, across drafters.
8. Treat the speculative setting as part of the accuracy configuration.
9. Compare builds on the same items and analyse them as paired; before reporting a null, state the smallest p the design could return.
10. For output-budgeted tasks with reasoning on, report how many responses finished reasoning.
11. For LLM-judged benchmarks, name the judge and its settings; use two judges and report agreement.
12. Save server logs before removing any container; give every run a unique label that includes every dimension that varies; make a run that measured nothing fail.
13. Before serving, soak-test a growing multi-turn conversation to the full window and watch host RAM and swap activity, not only VRAM.
14. Keep per-item records. Recompute headline numbers from them when writing up.